



Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering

ACADEMIC YEAR: 2025-26

(WINTER SEMESTER)

LAB RECORD

10211CS224-Full Stack Application Development

Name : Srimanthula Sravanthi

VTU No. : 24790

Reg. No. : 23UECS056

Branch : CSE

Year/Sem : 3 / 6

Slot : E3

10211CS224-FullStackApplicationDevelopment

TABLEOFCONTENTS

TaskNo	Date	ExperimentTitle
1.	05-01-2026	StudentRegistrationSystemwithDatabaseStorageandRetrieval
2.	22-01-2026	DataRetrieval,Sorting,and FilteringDashboard
3.	29-01-2026	LoginSystemwithInputValidationandAuthentication
4.	02-02-2026	OrderManagementUsingSQLJoinsandSubqueries
5.	05-02-2026	Transaction-BasedPaymentSimulationUsingCOMMITandROLLBACK
6.	09-02-2026	AutomatedLoggingUsingDatabaseTriggersandViews
7.	12-02-2026	InteractiveWebFormUsingJavaScriptEventsandFunctions
8.	16-02-2026	EmployeeManagementUsingSpringCoreandDependencyInjection
9.	19-02-2026	SpringMVCApplicationUsingAnnotation-BasedConfiguration
10.	23-02-2026	StudentCRUDOperationsUsingSpringBootandJPA
11.	26-02-2026	DataAccessUsingSpringDataJPAwithSortingandPagination
12.	02-03-2026	RESTfulAPIforProductManagement UsingSpringBoot
13.	05-03-2026	ExceptionHandlingandValidationinRESTfulApplications
14.	09-03-2026	MicroservicesArchitecturewithServiceDecomposition
15.	16-03-2026	ServiceRegistryandDiscoveryUsingEureka
16.	28-03-2026	APIGatewaywithLoadBalancingforMicroservices
17.	30-03-2026	Inter-ServiceCommunicationUsingRESTAPIs
18.	02-04-2026	UnitTestingforMicroservicesApplications
19.	06-04-2026	VersionControlUsingGitandRemoteRepositories
20.	08-04-2026	GitBranching,Merging,andConflictResolution
21.	13-04-2026	CI/CDPipeline forAutomatedBuildandDeployment
22.	16-04-2026	CloudServiceProvidersComparisonandPricingModels
23.	20-04-2026	PromptEngineering forCodeGenerationUsingGenerativeAI
24.	27-04-2026	Cloud-BasedApplicationUsingVibeCodingandGenerativeAI
25.	30-04-2026	UseCases

Task-1:Student Registration&DataStorageSystem

AIM:TodevelopaStudentRegistrationSystemthatalloowsuserstoenter,store,andmanage student details efficiently using a web interface and a MySQL database.

ALGORITHM:

Step1:Start

Step2:Openthewebapplication(indexpage)

Step3:Displayhomepagewithoptiontogotoregistrationpage Step

4: User navigates to the registration form

Step5:Userentersstudentdetails(likename,ID,course,etc.) Step 6:

Validate input fields

- Checkiffieldsareempty
- Ensurecorrect data format

Step 7: Submit the form data to the server

Step8:ServerreceivesthedatausingNode.js

Step9:EstablishconnectionwithMySQLdatabase

- Ifconnectionfails→displayerror
- Else→proceed

Step10:Insertstudentdataintodatabasetable

Step 11: Confirm successful storage of data

Step 12: Display success message to user

Step13:Allowusertoview/managestoreddata(ifimplemented)

Step 14: Stop

PROGRAM:

INDEX.HTML:

<!DOCTYPEhtml>

```
<html>
<head>
<title>StudentPortal</title>
<linkrel="stylesheet"href="style.css">
</head>
<body>
<divclass="home-container">
<h1>WelcometoStudentPortal</h1>
<p>Registerandmanagestudentinformationeasily.</p>
<a href="registration.html"class="start-btn">GotoRegistration</a>
</div>
</body>
</html>
```

REGISTRATION.HTML:

```
<!DOCTYPEhtml>
<html>
<head>
<title>StudentRegistration</title>
<linkrel="stylesheet"href="style.css">
</head>
<body>
<divclass="container">
<h1>StudentRegistration</h1>
<buttononclick="goHome()"class="home-btn">Home</button>
<formid="studentForm">
<label>Name</label>
```

```

<input type="text" id="name" placeholder="Enter your name" required>
</input>
<label>Name</label>
<input type="email" id="email" placeholder="Enter your email" required>
</input>
<label>Email</label>
<input type="date" id="dob" required>
</input>
<label>Date of Birth</label>
<input type="text" id="department" required>
</input>
<label>Department</label>
<select id="department" required>
  <option value="">Select Department</option>
  <option>CSE</option>
  <option>IT</option>
  <option>ECE</option>
  <option>MECH</option>
</select>
<label>Phone</label>
<input type="text" id="phone" placeholder="Enter phone number" required>
</input>
<button type="submit">Register</button>
</form>
</div>
<script src="script.js"></script>
</body>
</html>

```

STYLE.CSS:

```

body{
  font-family:'Segoe UI',sans-serif;
  background:linear-gradient(135deg,#0f2027,#203a43,#2c5364);
  height:100vh;
}

```

```
display:flex;
justify-content:center;
align-items:center;
margin:0;
}

.container{
width:420px;
padding:35px;
background:rgba(255,255,255,0.08);
border-radius:18px;
backdrop-filter:blur(10px);
box-shadow:0 10px 35px rgba(0,0,0,0.6);
}

h1{
text-align:center;
color:white;
margin-bottom:25px;
}

h2{
text-align:center;
color:white;
margin-top:30px;
}

form{
display:flex;
flex-direction:column;
```

```
}  
  
label{  
  color:#ddd;  
  font-size:14px;  
  margin-bottom:5px;  
  margin-top:10px;  
}  
  
input,select{  
  width:100%;  
  padding:12px;  
  border:none;  
  border-radius:8px;  
  background:#27496d;  
  color:white;  
  font-size:14px;  
  outline:none;  
  margin-bottom:5px;  
}  
  
input:focus,select:focus{  
  box-shadow:008px#00eaff;  
  background:#2c5d8a;  
}  
  
button{  
  margin-top:15px;  
  padding:12px;  
  border:none;
```

```
border-radius:10px;
background:linear-gradient(45deg,#ff4b2b,#ff416c);
color:white;
font-size:16px;
font-weight:bold;
cursor:pointer;
transition:0.3s;
}
button:hover{
transform:scale(1.05);
box-shadow:0 0 15px #ff416c;
}
#studentList{
margin-top:15px;
text-align:center;
color:white;
font-size:15px;
}
.home-container{
text-align:center;
color:white;
background:rgba(255,255,255,0.08);
padding:50px;
border-radius:20px;
backdrop-filter:blur(10px);
box-shadow:0 10px 35px rgba(0,0,0,0.6);
```

```
width:420px;
}

.home-containerh1{
font-size:32px;
margin-bottom:15px;
}

.home-container p{
margin-bottom:30px;
color:#ddd;
}

.start-btn{
text-decoration:none;
padding:12px 25px;
border-radius:10px;
background:linear-gradient(45deg,#ff4b2b,#ff416c);
color:white;
font-weight:bold;
transition:0.3s;
}

.start-btn:hover{
transform:scale(1.05);
box-shadow:0 0 15px #ff416c;
}
```

SCRIPT.JS:

```
const form=document.getElementById("studentForm")
```

```
form.addEventListener("submit",async(e)=>{
  e.preventDefault()
  conststudent={
    name: document.getElementById("name").value,
    email:document.getElementById("email").value,
    dob: document.getElementById("dob").value,
    department:document.getElementById("department").value,
    phone: document.getElementById("phone").value
  }
  try{
    constres=awaitfetch("http://localhost:3000/addStudent",{
      method:"POST",
      headers:{
        "Content-Type":"application/json"
      },
      body:JSON.stringify(student)
    })
    constdata=awaitres.text()
    alert(data)
    //clearform
    form.reset()
  }catch(error){
    console.log("Error:",error)
  }
})

functiongoHome(){
```

```
window.location.href="index.html"
}
```

SERVER.JS:

```
const express = require("express")
const cors = require("cors")
const path = require("path")
const db = require("../db")
const app = express()
app.use(cors())
app.use(express.json())
// Serve static files (HTML, CSS, JS)
app.use(express.static(__dirname))
// Redirect root link to index.html
app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "index.html"))
})
// Insert student data
app.post("/addStudent", (req, res) => {
  const { name, email, dob, department, phone } = req.body
  const sql = "INSERT INTO students(name,email,dob,department,phone) VALUES (?, ?, ?, ?, ?)"
  db.query(sql, [name, email, dob, department, phone], (err, result) => {
    if (err) {
      res.send(err)
    } else {
      res.send("Student Added Successfully")
    }
  })
})
```

```

    }
  })

  })

  // Get student data
  app.get("/students",(req,res)=>{
    db.query("SELECT*FROMstudents",(err,result)=>{
      if(err){
        res.send(err)
      }else{
        res.json(result)
      }
    })
  })

  })

  // Start server
  app.listen(3000,()=>{
    console.log("Serverrunningat:http://localhost:3000")
  })

```

DB.JS:

```

const mysql = require("mysql")

constdb=mysql.createConnection({
  host: "localhost",
  user: "root",
  password:"fahi",

```

```
database:"studentdb"

})

db.connect((err)=>{

if(err){

console.log("DatabaseError:",err)

}else{

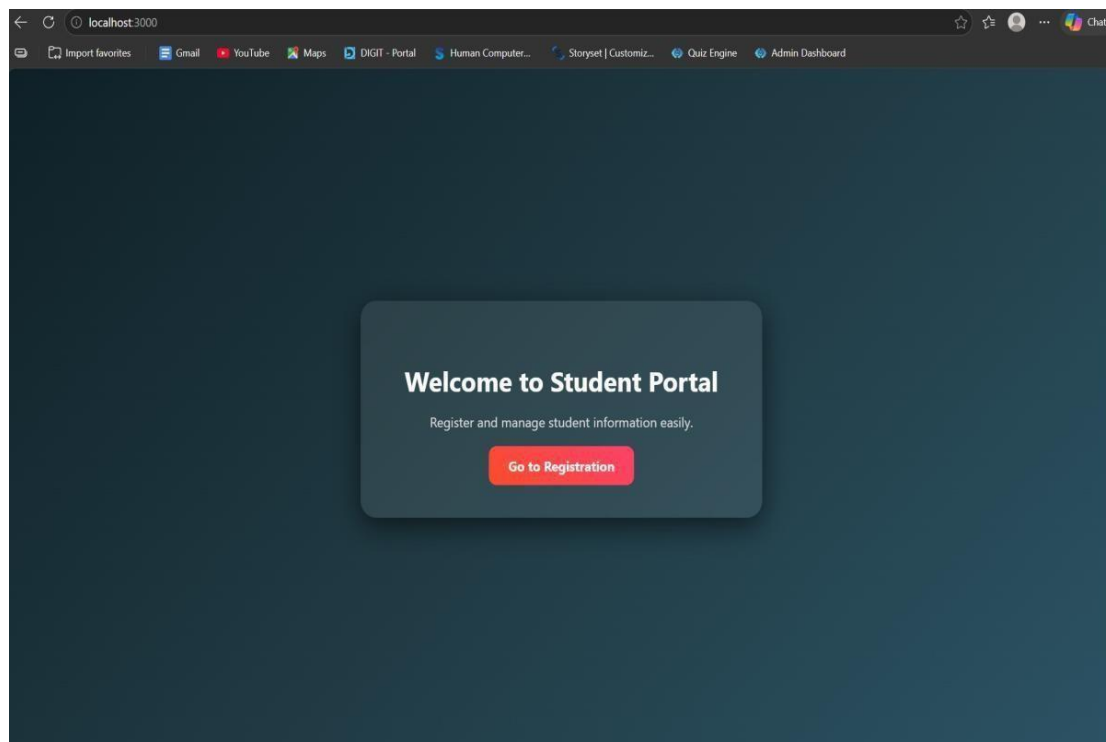
console.log("MySQLConnectedSuccessfully")

}

})

module.exports=db
```

OUTPUTS:



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/registration.html'. The browser's top bar includes navigation icons and a list of open tabs: 'Import favorites', 'Gmail', 'YouTube', 'Maps', 'DIGIT - Portal', 'Human Computer...', 'Storyset | Customiz...', 'Quiz Engine', and 'Admin Dashboard'. The main content area features a dark blue background with a central, lighter blue rounded rectangle containing the 'Student Registration' form. The form has a title 'Student Registration' at the top, followed by a red 'Home' button. Below this are five input fields: 'Name' (placeholder 'Enter your name'), 'Email' (placeholder 'Enter your email'), 'Date of Birth' (placeholder 'mm/dd/yyyy' with a calendar icon), 'Department' (a dropdown menu with 'Select Department'), and 'Phone' (placeholder 'Enter phone number'). At the bottom of the form is a red 'Register' button.

RESULT:The student registration system was successfully developed to store and manage student data using a web interface and MySQL database.

TASK2:DataRetrieval&SortingDashboard

AIM: To develop a web-based dashboard that allows users to store, retrieve, and display student data efficiently from a database, and present it in a structured format for easy access and management.

ALGORITHM:

1. Start the application
2. Initialize the server using Express framework
3. Establish database connection using MySQL
4. Load frontend pages (HTML, CSS, JS) in browser

► Data Insertion Process:

5. User enters student details (Name, Email, DOB, Department, Phone)
6. Send data to backend using POST request (/addStudent)
7. Execute SQL query to insert data into database
8. Display success message after insertion

► Data Retrieval Process:

9. Send GET request to server (/students)
10. Fetch all student records from database
11. Convert data into JSON format
12. Display data dynamically in dashboard table.
13. Arrange data in structured format (table/grid)
14. Allow better readability and management
15. Stop.

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

<title>StudentPortal</title>

<linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="home-container">

<h1>WelcometoStudentPortal</h1>

<p>Registerandmanagestudentinformationeasily.</p>

<a href="registration.html"class="start-btn">GotoRegistration</a>

<a href="dashboard.html"class="start-btn">ViewDashboard</a>

</div>

</body>

</html>
```

REGISTRATION.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

<title>StudentRegistration</title>

<linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="container">

<h1>StudentRegistration</h1>

<buttononclick="goHome()"class="home-btn">Home</button>

<formid="studentForm">
```

```
<label>Name</label>
<input type="text" id="name" placeholder="Enter your name" required>
<label>Email</label>
<input type="email" id="email" placeholder="Enter your email" required>
<label>Date of Birth</label>
<input type="date" id="dob" required>
<label>Department</label>
<select id="department" required>
  <option value="">Select Department</option>
  <option>CSE</option>
  <option>IT</option>
  <option>ECE</option>
  <option>MECH</option>
</select>
<label>Phone</label>
<input type="text" id="phone" placeholder="Enter phone number" required>
<button type="submit">Register</button>
</form>
</div>
<script src="script.js"></script>
</body>
</html>
```

DASHBOARD.HTML:

```
<!DOCTYPE html>
<html>
```

```
<head>

<title>StudentDashboard</title>

<linkrel="stylesheet"href="dashboard.css">

</head>

<body>

<divclass="container">

<buttononclick="goHome()"class="home-btn">Home</button>

<h1>StudentDashboard</h1>

<divclass="controls">

<selectid="sortSelect">

<optionvalue="">SortBy</option>

<optionvalue="name">Name</option>

<optionvalue="dob">DateofBirth</option>

</select>

<selectid="departmentFilter">

<optionvalue="">FilterbyDepartment</option>

<optionvalue="CSE">CSE</option>

<optionvalue="IT">IT</option>

<optionvalue="ECE">ECE</option>

<optionvalue="MECH">MECH</option>

</select>

<buttononclick="loadStudents()">Apply</button>

</div>

<table>

<thead>

<tr>
```

```
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>DOB</th>
<th>Department</th>
<th>Phone</th>
</tr>
</thead>
<tbodyid="studentTable"></tbody>
</table>
<h2>DepartmentCount</h2>
<divid="deptCount"></div>
</div>
<scriptsrc="dashboard.js"></script>
<script>
function goHome(){
window.location.href="index.html"
}
</script>
</body>
</html>
```

STYLE.CSS:

```
body{
font-family:'SegoeUI',sans-serif;
background:linear-gradient(135deg,#0f2027,#203a43,#2c5364);
```

```
height:100vh;  
display:flex;  
justify-content:center;  
align-items:center;  
margin:0;  
}
```

```
.container{  
width:420px;  
padding:35px;  
background:rgba(255,255,255,0.08);  
border-radius:18px;  
backdrop-filter:blur(10px);  
box-shadow:0 10px 35px rgba(0,0,0,0.6);  
}
```

```
h1{  
text-align:center;  
color:white;  
margin-bottom:25px;  
}
```

```
h2{  
text-align:center;  
color:white;  
margin-top:30px;
```

```
}
```

```
form{  
  display:flex;  
  flex-direction:column;  
}
```

```
label{  
  color:#ddd;  
  font-size:14px;  
  margin-bottom:5px;  
  margin-top:10px;  
}
```

```
input,select{  
  width:100%;  
  padding:12px;  
  border:none;  
  border-radius:8px;  
  background:#27496d;  
  color:white;  
  font-size:14px;  
  outline:none;  
  margin-bottom:5px;  
}
```

```
input:focus,select:focus{  
  box-shadow:008px#00eaff;  
  background:#2c5d8a;  
}
```

```
button{  
  margin-top:15px;  
  padding:12px;  
  border:none;  
  border-radius:10px;  
  background:linear-gradient(45deg,#ff4b2b,#ff416c);  
  color:white;  
  font-size:16px;  
  font-weight:bold;  
  cursor:pointer;  
  transition:0.3s;  
}
```

```
button:hover{  
  transform:scale(1.05);  
  box-shadow:0015px#ff416c;  
}
```

```
#studentList{  
  margin-top:15px;  
  text-align:center;
```

```
color:white;
font-size:15px;
}
.home-container{
text-align:center;
color:white;
background:rgba(255,255,255,0.08);
padding:50px;
border-radius:20px;
backdrop-filter:blur(10px);
box-shadow:0 10px35pxrgba(0,0,0,0.6);
width:420px;
}
```

```
.home-containerh1{
font-size:32px;
margin-bottom:15px;
}
```

```
.home-container p{
margin-bottom:30px;
color:#ddd;
}
```

```
.start-btn{
text-decoration:none;
```

```
padding:12px25px;
border-radius:10px;
background:linear-gradient(45deg,#ff4b2b,#ff416c);
color:white;
font-weight:bold;
transition:0.3s;
}
```

```
.start-btn:hover{
transform:scale(1.05);
box-shadow:0015px#ff416c;
}
```

DASHBOARD.CSS:

```
body{
font-family:SegoeUI;
background:linear-gradient(135deg,#0f2027,#203a43,#2c5364);
margin:0;
padding:0;
display:flex;
justify-content:center;
}
```

```
.container{
width:900px;
margin-top:40px;
```

```
background:rgba(255,255,255,0.08);  
padding:30px;  
border-radius:15px;  
backdrop-filter:blur(10px);  
color:white;  
}
```

```
h1{  
text-align:center;  
margin-bottom:20px;  
}
```

```
.controls{  
display:flex;  
gap:10px;  
margin-bottom:20px;  
}
```

```
select,button{  
padding:10px;  
border:none;  
border-radius:6px;  
}
```

```
button{  
background:#ff416c;
```

```
color:white;
cursor:pointer;
}
```

```
table{
width:100%;
border-collapse:collapse;
background:rgba(255,255,255,0.1);
}
```

```
th,td{
padding:10px;
text-align:center;
}
```

```
th{
background:#1f4068;
}
```

```
tr:nth-child(even){
background:rgba(255,255,255,0.05);
}
```

```
#deptCount{
margin-top:15px;
font-size:16px;
```

```
}  
  
.home-btn{  
padding:8px16px;  
border:none;  
border-radius:6px;  
background:#00c6ff;  
color:white;  
cursor:pointer;  
margin-bottom:10px;  
}
```

SCRIPT.JS:

```
const form=document.getElementById("studentForm")  
form.addEventListener("submit", async (e) => {  
  e.preventDefault()  
  const student={  
    name: document.getElementById("name").value,  
    email:document.getElementById("email").value,  
    dob: document.getElementById("dob").value,  
    department:document.getElementById("department").value,  
    phone: document.getElementById("phone").value  
  }  
  try{  
    const res=await fetch("http://localhost:3000/addStudent",{  
      method:"POST",  
      headers:{
```

```
"Content-Type":"application/json"
},
body:JSON.stringify(student)
})
constdata=awaitres.text()
alert(data)
//clearform
form.reset()
}catch(error){
console.log("Error:",error)
}
})
function goHome(){
window.location.href="index.html"
}
```

SERVER.JS:

```
constexpress=require("express")
const cors = require("cors")
constpath=require("path")
const db = require("./db")
const app = express()
app.use(cors())
app.use(express.json())
//Servestaticfiles(HTML,CSS,JS)
app.use(express.static(__dirname))
```

```
//Redirectrootlinktoindex.html
app.get("/", (req, res) => {
    res.sendFile(path.join(dirname, "index.html"))
})

// Insert student data
app.post("/addStudent",(req,res)=>{
    const{name,email,dob,department,phone}=req.body
    const sql= "INSERT INTO students(name,email,dob,department,phone) VALUES (?,?,,?,?)"
    db.query(sql,[name,email,dob,department,phone] ,(err,result)=>{
        if(err){
            res.send(err)
        }else{
            res.send("StudentAddedSuccessfully")
        }
    })

})

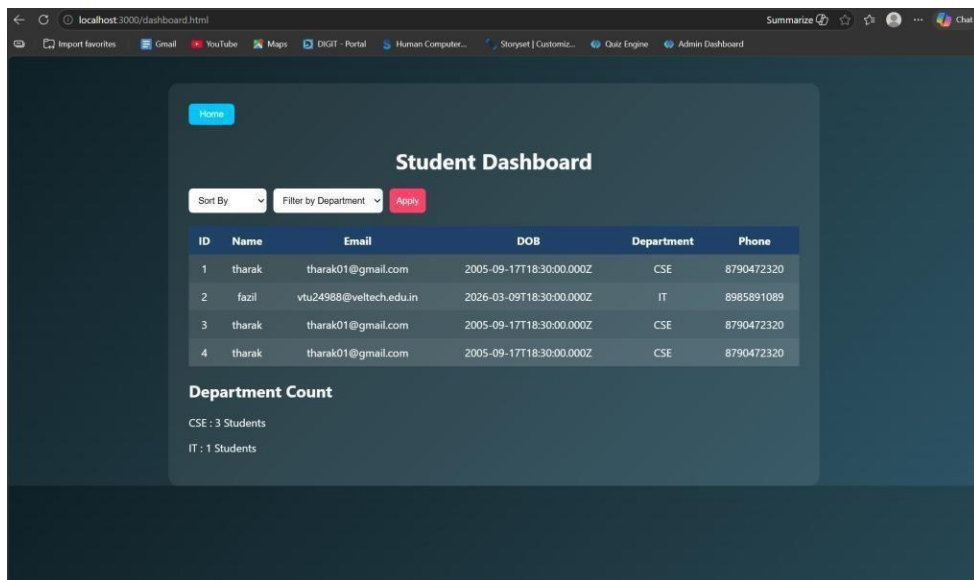
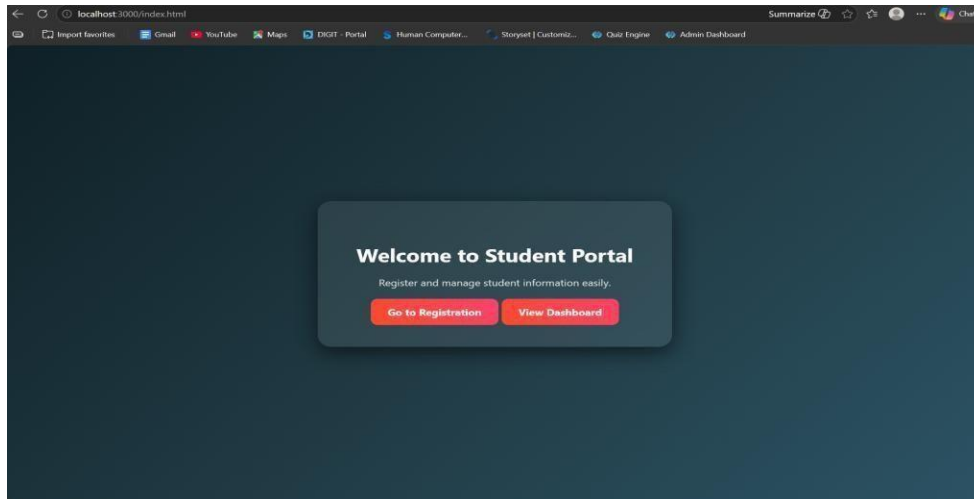
// Get student data
app.get("/students",(req,res)=>{
    db.query("SELECT*FROMstudents", (err,result)=>{
        if(err){
            res.send(err)
        }else{
            res.json(result)
        }
    })
})
```

```
})  
  
// Start server  
app.listen(3000,()=>{  
  console.log("Server running at: http://localhost:3000")  
})
```

DB.JS:

```
const mysql = require("mysql")  
const db = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "fahi",  
  database: "studentdb"  
})  
  
db.connect((err) => {  
  if (err) {  
    console.log("Database Error:", err)  
  } else {  
    console.log("MySQL Connected Successfully")  
  }  
})  
  
module.exports = db
```

OUTPUTS:



RESULT: The system successfully stores, retrieves, and displays student data efficiently through a user-friendly dashboard.

TASK3::LoginSystemwithValidation

AIM: To develop a secure login system that validates user credentials (username and password) and allows access only to authorized users.

ALGORITHM:

1. Start the application
2. Display login form (Username & Password fields)
3. User enters login credentials
4. Perform input validation:
 - Check if fields are empty
 - Validate email format (if applicable)
 - Ensure password meets required criteria
5. Send login data to backend server
6. Server receives request and processes input
7. Compare entered credentials with database records
8. If credentials match:
 - Grant access and redirect to dashboard
9. Else:
 - Display error message ("Invalid Username/Password")
10. End process

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

<title>StudentPortal</title>

<linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="home-container">

<h1>WelcometoStudentPortal</h1>

<p>Registerandmanagestudentinformationeasily.</p>

<a href="registration.html" class="start-btn">GotoRegistration</a>

<a href="dashboard.html" class="start-btn">ViewDashboard</a>

</div>

</body>

</html>
```

DASHBOARD.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

<title>StudentDashboard</title>

<linkrel="stylesheet"href="dashboard.css">

</head>

<body>

<divclass="container">

<buttononclick="goHome()" class="home-btn">Home</button>

<h1>StudentDashboard</h1>

<divclass="controls">
```

```
<selectid="sortSelect">
<optionvalue="">SortBy</option>
<optionvalue="name">Name</option>
<optionvalue="dob">DateofBirth</option>
</select>

<selectid="departmentFilter">
<optionvalue="">FilterbyDepartment</option>
<optionvalue="CSE">CSE</option>
<optionvalue="IT">IT</option>
<optionvalue="ECE">ECE</option>
<optionvalue="MECH">MECH</option>
</select>

<buttononclick="loadStudents()">Apply</button>
</div>

<table>
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>DOB</th>
<th>Department</th>
<th>Phone</th>
</tr>
</thead>
<tbodyid="studentTable"></tbody>
```

```
</table>
<h2>DepartmentCount</h2>
<div="deptCount"></div>
</div>
<scriptsrc="dashboard.js"></script>
<script>
function goHome(){
window.location.href="index.html"
}
</script>
</body>
</html>
```

LOGIN.HTML:

```
<!DOCTYPEhtml>
<html>
<head>
<title>Login</title>
<linkrel="stylesheet"href="login.css">
</head>
<body>
<divclass="login-container">
<h1>Login</h1>
<formid="loginForm">
<label>Username</label>
<inputtype="text" id="username">
```

```
<label>Password</label>
<input type="password" id="password">
<p id="errorMsg"></p>
<button type="submit">Login</button>
</form>
</div>
<script src="login.js"></script>
</body>
</html>
```

REGISTRATION.HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>StudentRegistration</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h1>StudentRegistration</h1>
<button onclick="goHome()" class="home-btn">Home</button>
<form id="studentForm">
<label>Name</label>
<input type="text" id="name" placeholder="Enter your name" required>
<label>Email</label>
<input type="email" id="email" placeholder="Enter your email" required>
```

```
<label>DateofBirth</label>
<inputtype="date"id="dob"required>
<label>Department</label>
<selectid="department"required>
<optionvalue="">SelectDepartment</option>
<option>CSE</option>
<option>IT</option>
<option>ECE</option>
<option>MECH</option>
</select>
<label>Phone</label>
<inputtype="text"id="phone"placeholder="Enterphonenumber" required>
<buttontype="submit">Register</button>
</form>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
font-family:'SegoeUI',sans-serif;
background:linear-gradient(135deg,#0f2027,#203a43,#2c5364);
height:100vh;
display:flex;
justify-content:center;
```

```
align-items:center;
margin:0;
}
```

```
.container{
width:420px;
padding:35px;
background:rgba(255,255,255,0.08);
border-radius:18px;
backdrop-filter:blur(10px);
box-shadow:0 10px 35px rgba(0,0,0,0.6);
}
```

```
h1{
text-align:center;
color:white;
margin-bottom:25px;
}
```

```
h2{
text-align:center;
color:white;
margin-top:30px;
}
```

```
form{
```

```
display:flex;
flex-direction:column;
}
```

```
label{
color:#ddd;
font-size:14px;
margin-bottom:5px;
margin-top:10px;
}
```

```
input,select{
width:100%;
padding:12px;
border:none;
border-radius:8px;
background:#27496d;
color:white;
font-size:14px;
outline:none;
margin-bottom:5px;
}
```

```
input:focus,select:focus{
box-shadow:008px#00eaff;
background:#2c5d8a;
```

```
}
```

```
button{  
margin-top:15px;  
padding:12px;  
border:none;  
border-radius:10px;  
background:linear-gradient(45deg,#ff4b2b,#ff416c);  
color:white;  
font-size:16px;  
font-weight:bold;  
cursor:pointer;  
transition:0.3s;  
}
```

```
button:hover{  
transform:scale(1.05);  
box-shadow:0 0 15px #ff416c;  
}
```

```
#studentList{  
margin-top:15px;  
text-align:center;  
color:white;  
font-size:15px;  
}
```

```
.home-container{  
text-align:center;  
color:white;  
background:rgba(255,255,255,0.08);  
padding:50px;  
border-radius:20px;  
backdrop-filter:blur(10px);  
box-shadow:0 10px35pxrgba(0,0,0,0.6);  
width:420px;  
}
```

```
.home-containerh1{  
font-size:32px;  
margin-bottom:15px;  
}
```

```
.home-container p{  
margin-bottom:30px;  
color:#ddd;  
}
```

```
.start-btn{  
text-decoration:none;  
padding:12px 25px;  
border-radius:10px;  
background:linear-gradient(45deg,#ff4b2b,#ff416c);
```

```
color:white;
font-weight:bold;
transition:0.3s;
}
```

```
.start-btn:hover{
transform:scale(1.05);
box-shadow:0015px#ff416c;
}
```

SERVER.JS:

```
const express = require("express")
const cors = require("cors")
const path = require("path")
const db = require("../db")
const app = express()
app.use(cors())
app.use(express.json())
app.use(express.urlencoded({extended:true}))
// Disable default index.html loading
app.use(express.static(dirname,{index:false}))
//=====
//OPENLOGINPAGEFIRST
//=====
app.get("/", (req, res) => {
res.sendFile(path.join(dirname,"login.html"))
```

```

})

//=====

//LOGINSYSTEM

//=====

app.post("/login",(req,res)=>{

const{username,password}=req.body

constsql="SELECT*FROMUsersWHEREUsername=?ANDpassword=?"db.query(sql,[usern
ame,password] ,(err,result)=>{

if(err){

res.send("error")

return

}

if(result.length>0){

res.send("success")

}else{

res.send("fail")

}

}}

//=====

//STUDENTREGISTRATION

//=====

app.post("/addStudent",(req,res)=>{

const{name,email,dob,department,phone}=req.body

const sql= "INSERT INTO students(name,email,dob,department,phone) VALUES (?,? ,?, ?,?)"

db.query(sql,[name,email,dob,department,phone] ,(err,result)=>{

```

```
if(err){
res.send(err)
}else{
res.send("StudentAddedSuccessfully")
}

})

})

//=====
//GETSTUDENTS(Dashboard)
//=====

app.get("/students",(req,res)=>{

db.query("SELECT*FROMstudents",(err,result)=>{

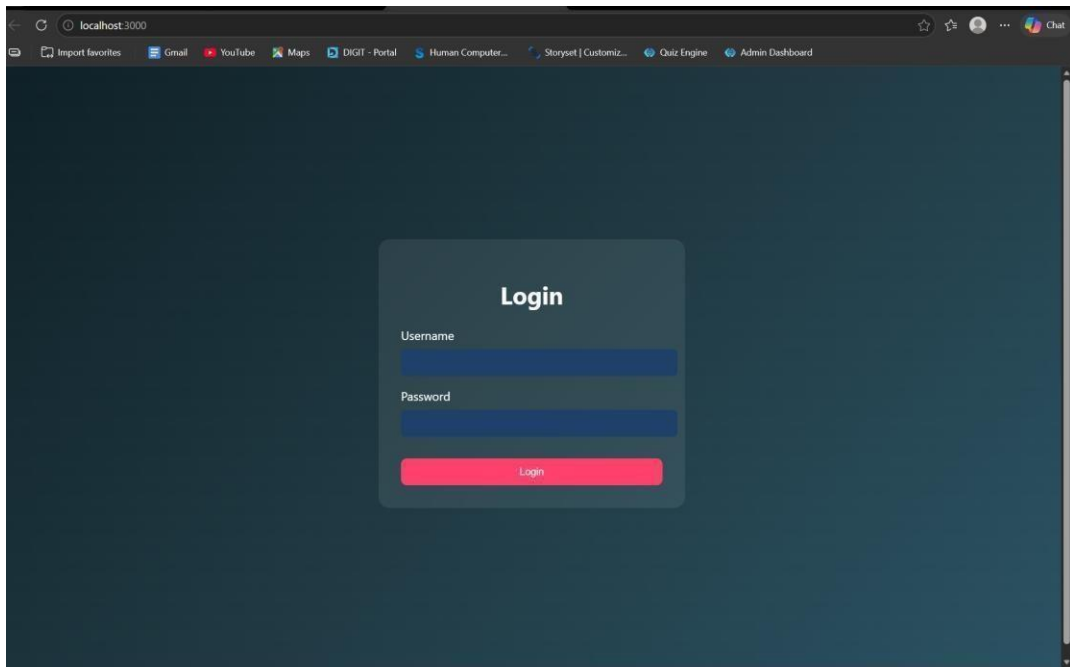
if(err){
res.send(err)
}else{
res.json(result)
}

})

})
```

```
//=====
//STARTSERVER
//=====
app.listen(3000,()=>{
  console.log("Server running at: http://localhost:3000")
})
```

OUTPUTS:



RESULT: The system successfully validates user credentials and grants access only to authorized users while preventing invalid logins.

TASK4:OrderManagementusingJoins

AIM: To design and implement an order management system using SQL joins and subqueries to efficiently retrieve and analyze customer, product, and order data.

ALGORITHM:

1. Start the process
2. Create database tables:
 - Customers(CustomerID, Name, etc.)
 - Products(ProductID, ProductName, Price)
 - Orders(OrderID, CustomerID, ProductID, Quantity, TotalAmount)
3. Insert sample data into all tables

► Data Retrieval using Joins:

4. Use SQL JOIN queries to combine data from Customers, Orders, and Products
5. Display customer order history with details (Name, Product, Quantity, TotalAmount)

► Subquery for Analysis:

6. Use subquery to find the highest value order
7. Use subquery with GROUP BY to find the most active customer (maximum number of orders)

► Sorting:

8. Apply ORDER BY clause to sort results (e.g., highest order value first)
9. Display results in structured format (table/UI with CSS if frontend used)
10. Stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

<title>Login</title>

<linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="login">

<h2>Admin Login</h2>

<inputtype="text"id="username"placeholder="Username">

<input type="password"id="password"placeholder="Password">

<buttononclick="login()">Login</button>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

ADMINDASHBOARD.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

<title>OrderDashboard</title>

<linkrel="stylesheet"href="style.css">
```

```
</head>

<body>

<!--NAVBAR-->

<divclass="navbar">

<h2>OrderManagement</h2>

<a href="index.html">

<buttonclass="logout-btn">Logout</button>

</a>

</div>

<h1>OrderManagementDashboard</h1>

<!--CARDSSECTION-->

<divclass="grid">

<!--AddCustomer-->

<divclass="card">

<h3>AddCustomer</h3>

<inputid="cname"placeholder="CustomerName">

<inputid="cemail"placeholder="Email">

<buttononclick="addCustomer()">Add Customer</button>

</div>

<!--AddProduct-->

<divclass="card">

<h3>AddProduct</h3>

<inputid="pname"placeholder="ProductName">

<inputid="pprice"placeholder="Price">

<buttononclick="addProduct()">AddProduct</button>

</div>
```

```
<!--PlaceOrder-->
<divclass="card">
<h3>PlaceOrder</h3>
<selectid="customerSelect">
<optionvalue="">SelectCustomer</option>
</select>
<selectid="productSelect">
<optionvalue="">SelectProduct</option>
</select>
<inputid="qty"placeholder="Quantity">
<buttononclick="placeOrder()">PlaceOrder</button>
</div>
</div>
<!--ACTIONBUTTONS-->
<divclass="actions">
<buttononclick="loadOrders()">OrderHistory</button>
<buttononclick="highestOrder()">HighestOrder</button>
<buttononclick="activeCustomer()">MostActive Customer</button>
</div>
<!--RESULTAREA-->
<divid="result"></div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
*{  
margin:0;  
padding:0;  
box-sizing:border-box;  
font-family:'SegoeUI',Tahoma,Geneva,Verdana,sans-serif;  
}
```

```
body{  
background:linear-gradient(135deg,#5f6caf,#9a6bff);  
min-height:100vh;  
display:flex;  
flex-direction:column;  
align-items:center;  
}
```

```
/*LOGINPAGE*/
```

```
.login{  
width:350px;  
padding:40px;  
margin-top:120px;  
background:white;  
border-radius:15px;  
box-shadow:0 10px25pxrgba(0,0,0,0.2);  
text-align:center;  
}
```

```
.loginh2{  
margin-bottom:20px;  
color:#333;  
}
```

```
.login input{  
width:100%;  
padding:12px;  
margin:10px 0;  
border:1pxsolid#ccc;  
border-radius:8px;  
outline:none;  
}
```

```
.login button{  
width:100%;  
padding:12px;  
border:none;  
background:#5f6caf;  
color:white;  
font-weight:bold;  
border-radius:8px;  
cursor:pointer;  
transition:0.3s;  
}
```

```
.loginbutton:hover{  
background:#7c4dff;  
}
```

```
/*DASHBOARDTITLE*/
```

```
h1{  
margin-top:30px;  
margin-bottom:25px;  
color:white;  
}
```

```
/*CARDGRID */
```

```
.grid{  
display:flex;  
gap:25px;  
flex-wrap:wrap;  
justify-content:center;  
margin-bottom:20px;  
}
```

```
/*CARDS*/
```

```
.card{
```

```
width:260px;
padding:22px;
border-radius:15px;
box-shadow:08px20pxrgba(0,0,0,0.2);
text-align:center;
transition:0.3s;
color:white;
}
```

```
/*Different cardcolors*/
```

```
.card:nth-child(1){
background:linear-gradient(135deg,#ff7a18,#ffb347);
}
```

```
.card:nth-child(2){
background:linear-gradient(135deg,#00c6ff,#0072ff);
}
```

```
.card:nth-child(3){
background:linear-gradient(135deg,#43e97b,#38f9d7);
color:#222;
}
```

```
.card:hover{
transform:translateY(-6px);
```

```
}
```

```
.cardh3{  
margin-bottom:10px;  
}
```

```
/*INPUTS*/
```

```
.card input{  
width:90%;  
padding:10px;  
margin:6px;  
border-radius:6px;  
border:none;  
}
```

```
/*CARDBUTTON*/
```

```
.card button{  
padding:10px15px;  
border:none;  
background:white;  
color:#333;  
border-radius:6px;  
cursor:pointer;  
font-weight:bold;
```

```
transition:0.3s;  
}
```

```
.cardbutton:hover{  
background:#f1f1f1;  
}
```

```
/*ACTIONBUTTONS*/
```

```
button{  
margin:10px;  
padding:12px18px;  
border:none;  
border-radius:6px;  
background:#ffffff;  
color:#333;  
font-weight:bold;  
cursor:pointer;  
transition:0.3s;  
}
```

```
button:hover{  
background:#eaeaea;  
}
```

```
/*TABLE*/
```

```
table{  
width:85%;  
margin:auto;  
margin-top:30px;  
border-collapse:collapse;  
background:white;  
border-radius:12px;  
overflow:hidden;  
box-shadow:05px15pxrgba(0,0,0,0.2);  
}
```

```
th{  
background:#5f6caf;  
color:white;  
padding:14px;  
font-size:18px;  
}
```

```
td{  
padding:12px;  
text-align:center;  
border-bottom:1pxsolid#ddd;  
color:#222;  
font-weight:500;  
}
```

```
tr:nth-child(even){  
background:#f7f7f7;  
}
```

```
tr:hover{  
background:#eef1ff;  
}
```

```
/*RESULTTEXT*/
```

```
#result{  
margin-top:20px;  
color:white;  
font-size:18px;  
text-align:center;  
}
```

```
/*NAVBAR */
```

```
.navbar{  
  
width:100%;  
display:flex;  
justify-content:space-between;  
align-items:center;  
padding:15px 40px;
```

```
background:#2c2f5c;
color:white;
box-shadow:04px10pxrgba(0,0,0,0.2);

}
```

```
.navbarh2{
font-weight:600;
}
```

```
.logout-btn{

background:#ff5252;
color:white;
border:none;
padding:10px16px;
border-radius:6px;
cursor:pointer;
font-weight:bold;

}
```

```
.logout-btn:hover{

background:#ff1744;
```

```
}  
  
.actions{  
  
display:flex;  
justify-content:center;  
gap:20px;  
margin-top:30px;  
flex-wrap:wrap;  
  
}  
  
.actions button{  
padding:12px20px;  
font-size:16px;  
border:none;  
border-radius:8px;  
background:#5f6caf;  
color:white;  
cursor:pointer;  
transition:0.3s;  
}  
  
.actionsbutton:hover{  
background:#7c4dff;  
transform:scale(1.05);  
  
}
```

SCRIPT.JS:

```
//LOGINFUNCTION
```

```
functionlogin(){
```

```
    constusername=document.getElementById("username").value
```

```
    const password=document.getElementById("password").value
```

```
    fetch("/login",{
```

```
        method:"POST",
```

```
        headers:{"Content-Type":"application/json"},
```

```
        body:JSON.stringify({username,password})
```

```
    })
```

```
    .then(res=>res.json())
```

```
    .then(data=>{
```

```
        if(data.success){
```

```
            window.location="admindashboard.html"
```

```
        }else{
```

```
            alert("Invalidlogin")
```

```
}  
})
```

```
}
```

```
//ADDCUSTOMER
```

```
functionaddCustomer(){
```

```
fetch("/add-customer",{
```

```
method:"POST",
```

```
headers:{"Content-Type":"application/json"},
```

```
body:JSON.stringify({
```

```
name:document.getElementById("cname").value,
```

```
email:document.getElementById("cemail").value
```

```
})
```

```
})
```

```
.then(res=>res.text())
```

```
.then(alert)
```

```
}
```

```
//ADDPRODUCT
```

```
functionaddProduct(){
```

```
  fetch("/add-product",{
```

```
    method:"POST",
```

```
    headers:{"Content-Type":"application/json"},
```

```
    body:JSON.stringify({
```

```
      product:document.getElementById("pname").value,
```

```
      price:document.getElementById("pprice").value
```

```
    })
```

```
  })
```

```
  .then(res=>res.text())
```

```
  .then(alert)
```

```
}
```

```
//LOADCUSTOMERSINTODROPDOWN
```

```
asyncfunctionloadCustomers(){

constres=awaitfetch("/customers")
const data = await res.json()

letselect=document.getElementById("customerSelect")

if(!select)return

select.innerHTML='<optionvalue="">SelectCustomer</option>'

data.forEach(c=>{

select.innerHTML+=`<optionvalue="${c.id}">${c.name}</option>`

})

}

//LOADPRODUCTSINTODROPDOWN

asyncfunctionloadProducts(){

constres=awaitfetch("/products")
```

```
const data = await res.json()
```

```
let select = document.getElementById("productSelect")
```

```
if(!select) return
```

```
select.innerHTML = '<option value="">Select Product</option>'
```

```
data.forEach(p => {
```

```
select.innerHTML += `<option value="${p.id}">${p.product_name}</option>`
```

```
})
```

```
}
```

```
//PLACEORDER
```

```
function placeOrder() {
```

```
fetch("/place-order", {
```

```
method: "POST",
```

```
headers: { "Content-Type": "application/json" },
```

```
body:JSON.stringify({  
  
customer_id:document.getElementById("customerSelect").value,  
product_id:document.getElementById("productSelect").value,  
quantity:document.getElementById("qty").value
```

```
}}
```

```
}}
```

```
.then(res=>res.text())  
.then(alert)
```

```
}
```

```
//ORDERHISTORY(JOIN)
```

```
asyncfunctionloadOrders(){
```

```
constres=awaitfetch("/orders")  
const data=await res.json()
```

```
let  
html=<"<table><tr><th>Customer</th><th>Product</th><th>Price</th><th>Qty</th><th>Total<  
/th></tr>"
```

```
data.forEach(o=>{
```

```
html+=`
```

```
<tr>
```

```
<td>${o.name}</td>
```

```
<td>${o.product_name}</td>
```

```
<td>${o.price}</td>
```

```
<td>${o.quantity}</td>
```

```
<td>${o.total}</td>
```

```
</tr>
```

```
,
```

```
})
```

```
html+="</table>"
```

```
document.getElementById("result").innerHTML=html
```

```
}
```

```
//HIGHESTORDER(SUBQUERY)
```

```
asyncfunctionhighestOrder(){

constres=awaitfetch("/highest-order")
const data=await res.json()

document.getElementById("result").innerHTML=

`<h2>HighestOrder</h2>

${data[0].name}- ₹${data[0].total}`

}
```

```
//MOSTACTIVECUSTOMER
```

```
asyncfunctionactiveCustomer(){

constres=awaitfetch("/active-customer")
const data=await res.json()

document.getElementById("result").innerHTML=

`<h2>MostActiveCustomer</h2>
```

```
${data[0].name}(${data[0].total_orders}orders)`
```

```
}
```

```
//AUTOLOADDROPDOWNSWHENPAGELOADS
```

```
window.onload=function(){
```

```
loadCustomers()
```

```
loadProducts()
```

```
}
```

SERVER.JS:

```
const express=require("express")
```

```
const db=require("./db")
```

```
const app=express()
```

```
app.use(express.json())
```

```
app.use(express.static(_dirname))
```

```
//LOGIN
```

```
app.post("/login",(req,res)=>{
```

```
const {username,password}=req.body
```

```
const sql="SELECT*FROMusersWHEREusername=?ANDpassword=?"
```

```
db.query(sql,[username,password] ,(err,result)=>{
```

```
if(err)throwerr
```

```
if(result.length>0){
```

```
res.json({success:true})
```

```
}else{
```

```
res.json({success:false})
```

```
}
```

```
})
```

```
})
```

```
//ADDCUSTOMER
```

```
app.post("/add-customer", (req,res)=>{
```

```
const {name,email}=req.body
```

```
const sql = "INSERT INTO customers(name,email)VALUES(?,?)"
```

```
db.query(sql,[name,email],(err)=>{
```

```
if(err)throw err
```

```
res.send("CustomerAddedSuccessfully")
```

```
})
```

```
})
```

```
//ADDPRODUCT
```

```
app.post("/add-product",(req,res)=>{
```

```
const {product,price}=req.body
```

```
const sql = "INSERT INTO products(product_name,price)VALUES(?,?)"
```

```
db.query(sql,[product,price],(err)=>{
```

```
if(err)throw err
```

```
res.send("ProductAddedSuccessfully")
```

```
}}
```

```
}}
```

```
//GETCUSTOMERS(fordropdown)
```

```
app.get("/customers",(req,res)=>{
```

```
constsql="SELECT*FROMcustomers"
```

```
db.query(sql,(err,result)=>{
```

```
if(err)throwerr
```

```
res.json(result)
```

```
}}
```

```
}}
```

```
//GETPRODUCTS(fordropdown)
```

```
app.get("/products",(req,res)=>{
```

```
const sql = "SELECT * FROM products"
```

```
db.query(sql, (err, result) => {
```

```
  if (err) throw err
```

```
  res.json(result)
```

```
})
```

```
})
```

```
// PLACE ORDER
```

```
app.post("/place-order", (req, res) => {
```

```
  const { customer_id, product_id, quantity } = req.body
```

```
  const sql = "INSERT INTO orders (customer_id, product_id, quantity) VALUES (?, ?, ?)"
```

```
  db.query(sql, [customer_id, product_id, quantity], (err) => {
```

```
    if (err) throw err
```

```
    res.send("Order Placed Successfully")
```

```
}}
```

```
}}
```

```
//ORDERHISTORY(JOIN)
```

```
app.get("/orders",(req,res)=>{
```

```
constsql=`
```

```
SELECTcustomers.name,
```

```
products.product_name,
```

```
products.price,
```

```
orders.quantity,
```

```
(products.price*orders.quantity)AStotal
```

```
FROMorders
```

```
JOIN customers
```

```
ONorders.customer_id=customers.id
```

```
JOIN products
```

```
ONorders.product_id= products.id
```

```
ORDER BYorders.order_dateDESC
```

```
,
```

```
db.query(sql,(err,result)=>{
```

```
if(err)throwerr
```

```
res.json(result)
```

```
})
```

```
})
```

```
//HIGHESTVALUEORDER(SUBQUERY)
```

```
app.get("/highest-order",(req,res)=>{
```

```
constsql=`
```

```
SELECTcustomers.name,
```

```
(products.price*orders.quantity)AStotal
```

```
FROMorders
```

```
JOIN customers
```

```
ONorders.customer_id=customers.id
```

```
JOIN products
```

```
ONorders.product_id= products.id
```

```
WHERE(products.price*orders.quantity)=
```

```
(
```

```
SELECTMAX(products.price*orders.quantity)
```

```
FROM orders
```

```
JOIN products
```

```
ONorders.product_id= products.id
```

```
)
```

```
,
```

```
db.query(sql,(err,result)=>{
```

```
if(err)throwerr
```

```
res.json(result)
```

```
})
```

```
})
```

```
//MOSTACTIVECUSTOMER
```

```
app.get("/active-customer",(req,res)=>{
```

```
const sql = `
```

```
SELECT customers.name,  
COUNT(orders.id) AS total_orders
```

```
FROM customers
```

```
JOIN orders
```

```
ON customers.id = orders.customer_id
```

```
GROUP BY customers.id
```

```
ORDER BY total_orders DESC
```

```
LIMIT 1
```

```
`
```

```
db.query(sql, (err, result) => {
```

```
  if (err) throw err
```

```
  res.json(result)
```

```
})
```

```
}}
```

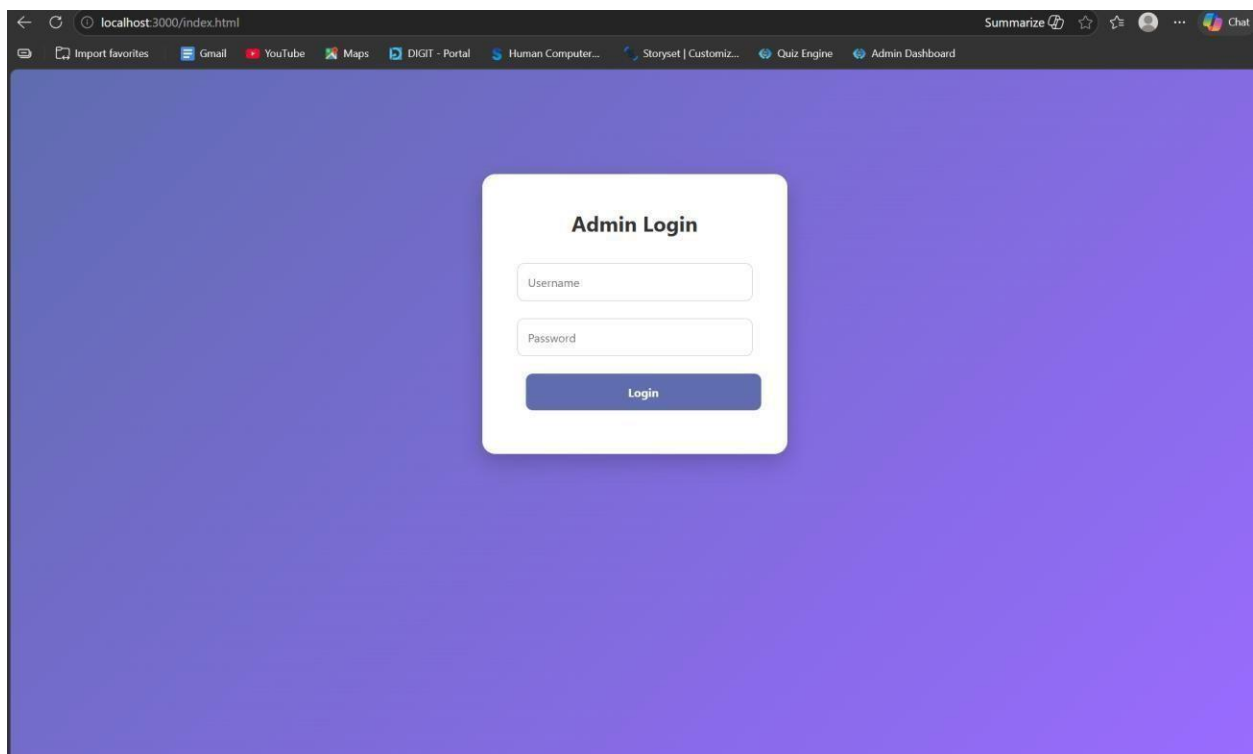
```
//STARTSERVER
```

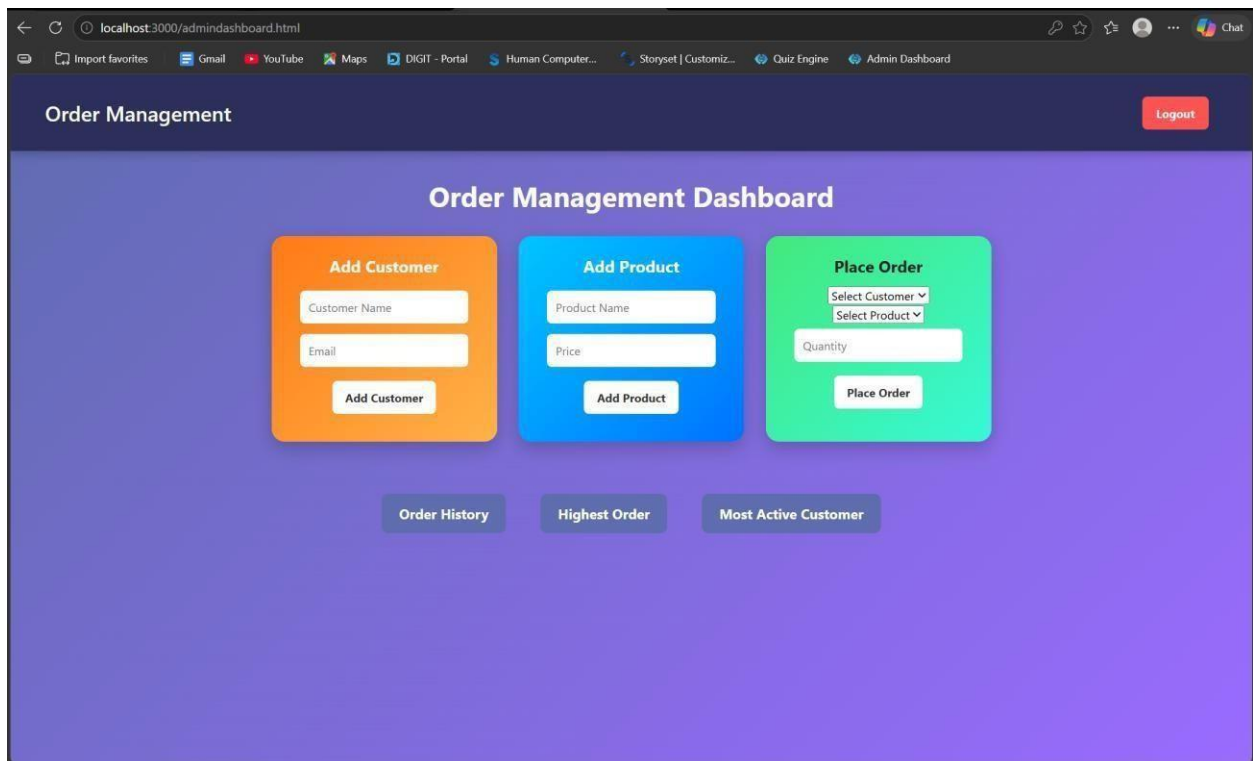
```
app.listen(3000,()=>{
```

```
console.log("Server running at http://localhost:3000")
```

```
}}
```

OUTPUTS:





RESULT: The system successfully retrieves and analyzes customer order data using joins and subqueries, displaying meaningful insights like highest order value and most active customer.

TASK5:Transaction-BasedPaymentSimulation

AIM: Tosimulateanonlinepaymentsystemusingdatabasetransactionsthatensuressafeand reliable fund transfer between user and merchant accounts using COMMIT and ROLLBACK operations.

ALGORITHM:

1. Starttheprocess
2. Createdatabasetables:
 - UserAccount(UserID,Balance)
 - MerchantAccount(MerchantID,Balance)
3. Insertinitialbalancevaluesforuserandmerchant

►TransactionProcess:

4. Begintransaction(STARTTRANSACTION)
5. Deductamountfromuseraccount
6. Addamounttomerchantaccount

►Validation:

7. Checkifuserhassufficientbalance
 - Ifyes→proceed
 - Ifno→gotorollback

►Commit/Rollback:

8. Ifalloperationsaresuccessful→COMMITtransaction
9. Ifanyerroroccurs→ROLLBACKtransaction
10. Displaytransactionstatus(Success/Failed)
11. Stop

PROGRAM:**INDEX.HTML:**

```
<!DOCTYPEhtml>

<html>

<head>

<title>PaymentSimulation</title>

<linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="container">

<h2>OnlinePayment</h2>

<selectid="user"onchange="updateBalances()"></select>

<selectid="merchant"onchange="updateBalances()"></select>

<inputtype="number"id="amount"placeholder="Enteramount">

<buttononclick="pay()">Pay</button>

<h3id="userBalance"></h3>

<h3id="merchantBalance"></h3>

<pid="result"></p>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```
body{

font-family:Arial;
```

```
background:linear-gradient(135deg,#dfe9f3,#ffffff);  
display:flex;  
justify-content:center;  
align-items:center;  
height:100vh;  
}
```

```
.container{  
background:white;  
padding:30px;  
border-radius:15px;  
box-shadow:0 10px25pxrgba(0,0,0,0.2);  
text-align:center;  
width:400px;  
}
```

```
select, input{  
padding:10px;  
margin:10px;  
width:90%;  
border-radius:5px;  
border:1pxsolid#ccc;  
}
```

```
button{  
padding:10px;
```

```
width:95%;  
background:green;  
color:white;  
border:none;  
border-radius:5px;  
cursor:pointer;  
font-size:16px;  
}
```

```
button:hover{  
background:darkgreen;  
}
```

```
h3{  
margin:5px;  
color:#333;  
}
```

SCRIPT.JS:

```
letusersData=[]  
letmerchantsData=[]  
function loadData(){  
fetch("/balances")  
.then(res=>res.json())  
.then(data=>{  
usersData=data.users
```

```

merchantsData=data.merchants

letuserSelect=document.getElementById("user")

letmerchantSelect=document.getElementById("merchant")

userSelect.innerHTML=""

merchantSelect.innerHTML=""

// fill users

usersData.forEach(u=>{

userSelect.innerHTML+=`<optionvalue="${u.id}">${u.name}</option>`

})

window.onload =loadData

// fill merchants

merchantsData.forEach(m=>{

merchantSelect.innerHTML+=`<optionvalue="${m.id}">${m.name}</option>`

})

//showinitialbalances

updateBalances()

})

}

functionupdateBalances(){

letuserId=document.getElementById("user").value

letmerchantId=document.getElementById("merchant").value let

user = usersData.find(u=>u.id == userId)

letmerchant=merchantsData.find(m=>m.id==merchantId)

document.getElementById("userBalance").innerText =

"User Balance: ₹" + user.balance

document.getElementById("merchantBalance").innerText=

```

```

"ShopBalance:₹"+merchant.balance
}

functionpay(){
letuserId=document.getElementById("user").value
letmerchantId=document.getElementById("merchant").value let
amount = document.getElementById("amount").value
fetch("/pay",{
method:"POST",
headers:{
"Content-Type":"application/json"
},
body:JSON.stringify({
userId:userId,
merchantId:merchantId,
amount:amount
})
})
.then(res=>res.json())
.then(data=>{
document.getElementById("result").innerText = data.message
loadData()
})
}

window.onload =loadData

```

SERVER.JS:

```
const express = require("express")
const db = require("../db")
const app = express()
app.use(express.json())
app.use(express.static(_dirname))
app.get("/", (req, res) => {
  res.sendFile(dirname + "/index.html")
})
app.post("/pay", (req, res) => {
  const { userId, merchantId, amount } = req.body
  const amt = parseInt(amount)
  if (!amt || amt <= 0) {
    return res.json({ message: "Enter valid amount" })
  }
  db.beginTransaction(err => {
    if (err) return res.status(500).send("Transaction error")
    // 1. Get user balance
    db.query("SELECT balance FROM Users WHERE id=?", [userId], (err, result) => {
      if (err || result.length === 0) {
        return db.rollback(() => res.send("User not found"))
      }
      let balance = result[0].balance
      if (balance < amt) {
        return db.rollback(() => {
          res.json({ message: "Insufficient Balance" })
        })
      }
    })
  })
})
```

```
    })  
  }  
  

```

```
//2.Deductfromuser
```

```
db.query(  
  "UPDATEUsersSETbalance=balance-?WHEREid=?", [amt,  
    userId],  
  (err)=>{  
  
    if(err){  
      returndb.rollback(()=>res.send("Deductionfailed"))  
    }  
  

```

```
//3.Addtomerchant
```

```
db.query(  
  "UPDATEmerchantsSETbalance=balance+?WHEREid=?", [amt,  
    merchantId],  
  (err)=>{  
  
    if(err){  
      returndb.rollback(()=>res.send("Merchantupdatefailed"))  
    }  
  

```

```
// 4. Commit
```

```
db.commit(err=>{
```

```
  if(err){
```

```

        return db.rollback(() => res.send("Commit failed"))
    }

    res.json({message: "Payment Successful" })
  })
}
)
}
)
})
})
})
app.get("/balances", (req, res) => {

```

```

  db.query("SELECT * FROM users", (err, users) => {

```

```

    if (err) return res.send("Error fetching users")

```

```

  db.query("SELECT * FROM merchants", (err, merchants) => {

```

```

    if (err) return res.send("Error fetching merchants")

```

```

    res.json({
      users: users,
      merchants: merchants
    })

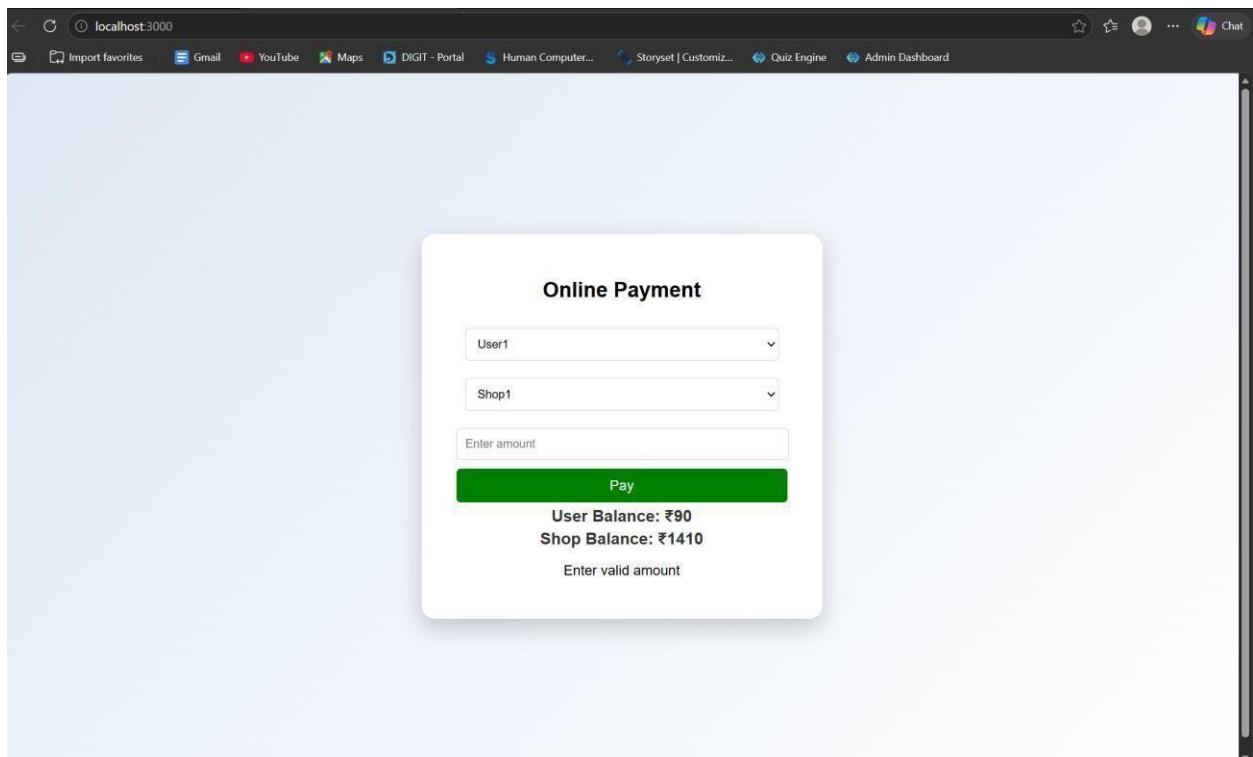
```

```

    })
  })
}
app.listen(3000,()=>{
  console.log("Server running on http://localhost:3000")
})

```

OUTPUTS:



The screenshot shows a web browser window with the address bar displaying 'localhost:3000'. The browser's tab bar includes links to 'Gmail', 'YouTube', 'Maps', 'DIGIT - Portal', 'Human Computer...', 'Storyset | Customiz...', 'Quiz Engine', and 'Admin Dashboard'. The main content area features a light blue background with a central white card titled 'Online Payment'. Inside the card, there are two dropdown menus labeled 'User1' and 'Shop1', followed by a text input field labeled 'Enter amount'. Below the input field is a prominent green button labeled 'Pay'. Underneath the button, the current balances are shown: 'User Balance: ₹90' and 'Shop Balance: ₹1410'. At the bottom of the card, the text 'Enter valid amount' is displayed.

RESULT: The system successfully performs secure payment transactions by updating balances accurately using COMMIT on success and ROLLBACK on failure.

TASK6:AutomatedLoggingusingTriggers&Views

AIM: To develop a web-based application for automated logging of database activities using MySQLtriggersandviews,integrated witha Node.jsbackendandfrontend interface,inorderto track INSERT and UPDATE operations and generate daily activity reports.

ALGORITHM:

1. Starttheprogram
2. Createdatabaseandrequired tables
3. CreatetriggersforINSERTandUPDATEoperations
4. Storeactionsinlogtableautomatically
5. Createaviewfordailyactivityreport
6. DevelopbackendusingNode.jsandExpress
7. ConnectbackendwithMySQLdatabase
8. CreateAPIsforadd,update,logs,andreport
9. DesignfrontendusingHTML,CSS,andJavaScript
10. Runtheapplicationanddisplayresults
11. Stoptheprogram

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

  <title>EmployeeAuditSystem</title>

  <linkrel="stylesheet"href="style.css">

</head>

<body>
```

```
<h2>EmployeeManagement</h2>
<divclass="form">
  <inputtype="number"id="emp_id"placeholder="EmployeeID">
  <inputtype="text"id="name"placeholder="Name">
  <inputtype="number"id="salary"placeholder="Salary">
  <buttononclick="addEmployee()">Add</button>
</div>
<divclass="form">
  <input type="number"id="update_id"placeholder="EmployeeID">
  <input type="number"id="new_salary"placeholder="NewSalary">
  <buttononclick="updateEmployee()">Update</button>
</div>
<buttononclick="loadLogs()">ViewLogs</button>
<buttononclick="loadReport()">ViewDailyReport</button>
<h3>Output</h3>
<preid="output"></pre>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family:Arial;
  text-align: center;
  background:#f4f4f4;
}
```

```
.form{
  margin:15px;
}
input{
  padding:8px;
  margin: 5px;
}
button{
  padding:8px15px;
  background:blue;
  color: white;
  border: none;
  cursor:pointer;
}
button:hover {
  background:darkblue;
}
pre{
  background:white;
  padding: 10px;
  width: 60%;
  margin: auto;
  text-align:left;
}
```

SCRIPT.JS:

```
constBASE_URL="http://localhost:3000";
```

```

// Add Employee
function addEmployee(){
    const data = {
        emp_id: document.getElementById("emp_id").value,
        name: document.getElementById("name").value,
        salary: document.getElementById("salary").value
    };
    fetch(`${BASE_URL}/add`, {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify(data)
    })
    .then(res => res.text())
    .then(data => alert(data));
}

// Update Employee
function updateEmployee(){
    const id = document.getElementById("update_id").value;
    const salary = document.getElementById("new_salary").value;

    fetch(`${BASE_URL}/update/${id}`, {
        method: "PUT",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({ salary })
    })
    .then(res => res.text())

```

```

        .then(data=>alert(data));
    }

    // Load Logs

    functionloadLogs(){

        fetch(`${BASE_URL}/logs`)

        .then(res=>res.json())

        .then(data => {

            document.getElementById("output").innerText=

                JSON.stringify(data,null,2);

        });
    }

    // Load Report

    functionloadReport(){

        fetch(`${BASE_URL}/report`)

        .then(res=>res.json())

        .then(data => {

            document.getElementById("output").innerText=

                JSON.stringify(data,null,2);

        });
    }

```

DB.JS:

```

const mysql = require('mysql2');

constdb=mysql.createConnection({

    host:'localhost',

    user: 'root',

    password:'fahi',

```

```

    database:'audit_db'
  });
  db.connect((err)=>{
    if (err) {
      console.error('DBConnectionFailed:',err);
    }else{
      console.log('ConnectedtoMySQL');
    }
  });
  module.exports=db;

```

OUTPUTS:

Employee Management

Employee ID: Name: Salary: [Add](#)

Employee ID: New Salary: [Update](#)

[View Logs](#) [View Daily Report](#)

Output

```

[
  {
    "log_id": 1,
    "emp_id": 1,
    "action_type": "INSERT",
    "action_time": "2026-04-13T04:59:54.000Z"
  },
  {
    "log_id": 2,
    "emp_id": 1,
    "action_type": "UPDATE",
    "action_time": "2026-04-13T05:00:36.000Z"
  }
]

```

RESULT: The system successfully logs all INSERT and UPDATE operations automatically using triggers and displays daily activity reports through a web interface.

TASK7:InteractiveWebFormwith Events&Functions

AIM:Todevelopaninteractiveweb-basedfeedbackformwithreal-timevalidation,Uleffects, and backend data storage using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Starttheprogram
2. Createprojectfiles(HTML,CSS,JS,server.js,db.js)
3. Designfeedbackformwithinputfields(name,email,message)
4. ApplyCSSstylingforlayoutandvisualeffects
5. Definereusablevalidationfunctions inJavaScript
6. CaptureuserinputusingDOMelements
7. Applykeyupeventforreal-timevalidation
8. Addhovereffecttohighlightinputfields
9. Implementdouble-clickeventonsubmitbutton
10. Validateallfieldsbeforesubmission
11. SenddatatoserverusingfetchAPI(POSTrequest)
12. Storedataindatabasemoduleanddisplaysuccessmessage

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<htmllang="en">

<head>

  <title>FeedbackForm</title>

  <linkrel="stylesheet"href="style.css">

</head>
```

```
<body>
<divclass="container">

  <h2>    CustomerFeedback</h2>

  <formid="feedbackForm">

    <inputtype="text"id="name"placeholder="YourName"required>

    <spanclass="error"id="nameError"></span>

    <inputtype="email"id="email"placeholder="YourEmail"required>

    <spanclass="error"id="emailError"></span>

    <textareaid="message"placeholder="YourFeedback"required></textarea>

    <spanclass="error"id="msgError"></span>

    <buttontype="button"id="submitBtn">Submit</button>

  </form>

  <pid="successMsg"></p>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```
body{

  font-family:'SegoeUI',sans-serif;

  background:linear-gradient(135deg,#1f1c2c,#928dab); display:

  flex;

  justify-content:center;

  align-items: center;

  height:100vh;

  color:white;
```

```
}  
  
.container{  
    background:rgba(255,255,255,0.1);  
    padding: 25px;  
    border-radius: 15px;  
    backdrop-filter:blur(10px);  
    width: 320px;  
    box-shadow:0 10px 25px rgba(0,0,0,0.3);  
}  
  
h2{  
    text-align:center;  
}  
  
input, textarea {  
    width: 100%;  
    padding:10px;  
    margin: 8px 0;  
    border-radius:8px;  
    border: none;  
    outline: none;  
    transition: 0.3s;  
}  
  
input:hover,textarea:hover{  
    transform: scale(1.05);  
    box-shadow:0 0 8px #00f7ff;  
}  
  
button{
```

```
width: 100%;
padding:10px;
background:#00c6ff;
border: none;
border-radius:10px;
color: white;
font-weight:bold;
cursor: pointer;
}
button:hover {
    background:#0072ff;
}
.error {
    color:#ff6b6b;
    font-size:12px;
}
#successMsg{
    text-align:center;
    margin-top:10px;
    color: #00ffae;
}
```

SCRIPT.JS:

```
const nameInput = document.getElementById("name");
const emailInput = document.getElementById("email");
const messageInput=document.getElementById("message");
const submitBtn = document.getElementById("submitBtn");
```

```

function validateName(){
    if (nameInput.value.length < 3) {
        document.getElementById("nameError").innerText="Min3characters";
        return false;
    }
    document.getElementById("nameError").innerText="";
    return true;
}

function validateEmail(){
    let pattern=/^[^]+@[^]+\.[a-z]{2,3}$/; if
    (!emailInput.value.match(pattern)) {
        document.getElementById("emailError").innerText="Invalidemail"; return
        false;
    }
    document.getElementById("emailError").innerText="";
    return true;
}

function validateMessage() {
    if (messageInput.value.length < 5) {
        document.getElementById("msgError").innerText="Tooshort";
        return false;
    }
    document.getElementById("msgError").innerText="";
    return true;
}

nameInput.addEventListener("keyup",validateName);

```

```

emailInput.addEventListener("keyup",validateEmail);
messageInput.addEventListener("keyup",validateMessage);
submitBtn.addEventListener("dblclick", () => {
    if(validateName()&&validateEmail()&&validateMessage()){
        sendData();
    }else{
        alert("Fixerrorsbeforesubmitting!");
    }
});
function sendData() {
    fetch("http://localhost:3000/submit",{
        method:"POST",
        headers: {
            "Content-Type":"application/json"
        },
        body: JSON.stringify({
            name:nameInput.value,
            email:emailInput.value,
            message:messageInput.value
        })
    })
    .then(res=>res.json())
    .then(data=>{
        document.getElementById("successMsg").innerText="    FeedbackSubmitted!";
    });
}

```

SERVER.JS:

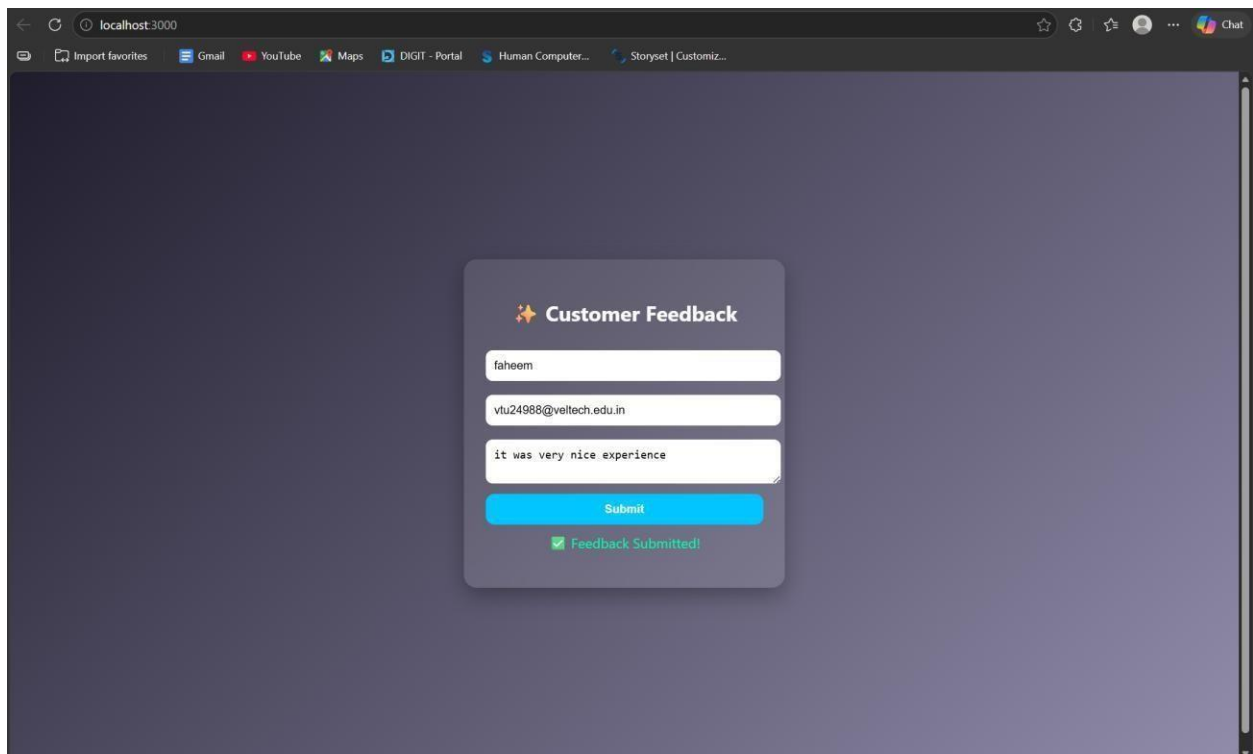
```
const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
const path = require("path");
const db = require("../db");
const app = express();
app.use(cors());
app.use(bodyParser.json());
app.use(express.static(__dirname));
app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "index.html"));
});
app.post("/submit", (req, res) => {
  const { name, email, message } = req.body;
  db.saveFeedback({ name, email, message });
  res.json({ status: "success" });
});
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

DB.JS:

```
let feedbacks = [];
function saveFeedback(data) {
  feedbacks.push(data);
  console.log("Saved:", data);
}
```

```
}  
  
module.exports={  
  saveFeedback  
};
```

OUTPUT:



RESULT:The interactive feedback form was successfully implemented with real-time validation, enhanced UI effects, and backend data handling.

TASK8:EmployeeManagement

AIM: To develop a simple Employee Management system using HTML, CSS, JavaScript, and Node.js for adding and displaying employee data.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design the user interface with input fields (name, role)
4. Apply CSS styling for layout and centering
5. Capture user input using JavaScript
6. Create function to add employee data
7. Send data to server using fetch API (POST request)
8. Create backend using Node.js and Express
9. Receive and store employee data in memory (db.js)
10. Create API to fetch all employee records (GET request)
11. Display employee list dynamically on the webpage
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

    <title>EmployeeManagement</title>

    <linkrel="stylesheet"href="style.css">

</head>
```

```
<body>
<divclass="container">
  <h2>EmployeeManagement</h2>
  <inputid="name"placeholder="Name">
  <inputid="role"placeholder="Role">
  <buttononclick="addEmployee()">Add</button>
  <ulid="list"></ul>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background:#1e1e2f;
  color: white;
  display:flex;
  justify-content:center;
  align-items: center;
  height:100vh;margin:
  0;
}
.container{
  text-align:center;
}
```

```
input{  
  padding:10px;  
  margin: 5px;  
  border-radius:5px;  
  border: none;  
}
```

```
button {  
  padding:10px;  
  background:#00c6ff;  
  border: none;  
  border-radius:5px;  
  cursor: pointer;  
}
```

SCRIPT.JS:

```
functionaddEmployee(){  
  constname=document.getElementById("name").value; const  
  role = document.getElementById("role").value;  
  fetch("http://localhost:3000/add", {  
    method:"POST",  
    headers: {  
      "Content-Type":"application/json"  
    },  
    body:JSON.stringify({name,role})  
  })  
  .then(res=>res.json())
```

```

        .then(()=>loadEmployees());
    }
function loadEmployees() {
    fetch("http://localhost:3000/all")
        .then(res=>res.json())
        .then(data=>{
            letlist=document.getElementById("list"); list.innerHTML
            = "";

            data.forEach(emp=>{
                let li = document.createElement("li");
                li.innerText=emp.name+"-"+emp.role;
                list.appendChild(li);
            });
        });
}
loadEmployees();

```

SERVER.JS:

```

constexpress=require("express");
const cors = require("cors");
constpath=require("path");
const db = require("./db");
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(dirname));

```

```

app.get("/",(req,res)=>{
    res.sendFile(path.join(dirname,"index.html"));
});
app.post("/add",(req,res)=>{
    db.addEmployee(req.body);
    res.json({ msg: "Added" });
});
app.get("/all", (req, res) => {
    res.json(db.getEmployees());
});
app.listen(3000,()=>{
    console.log("•   Serverrunningat:http://localhost:3000");
});

```

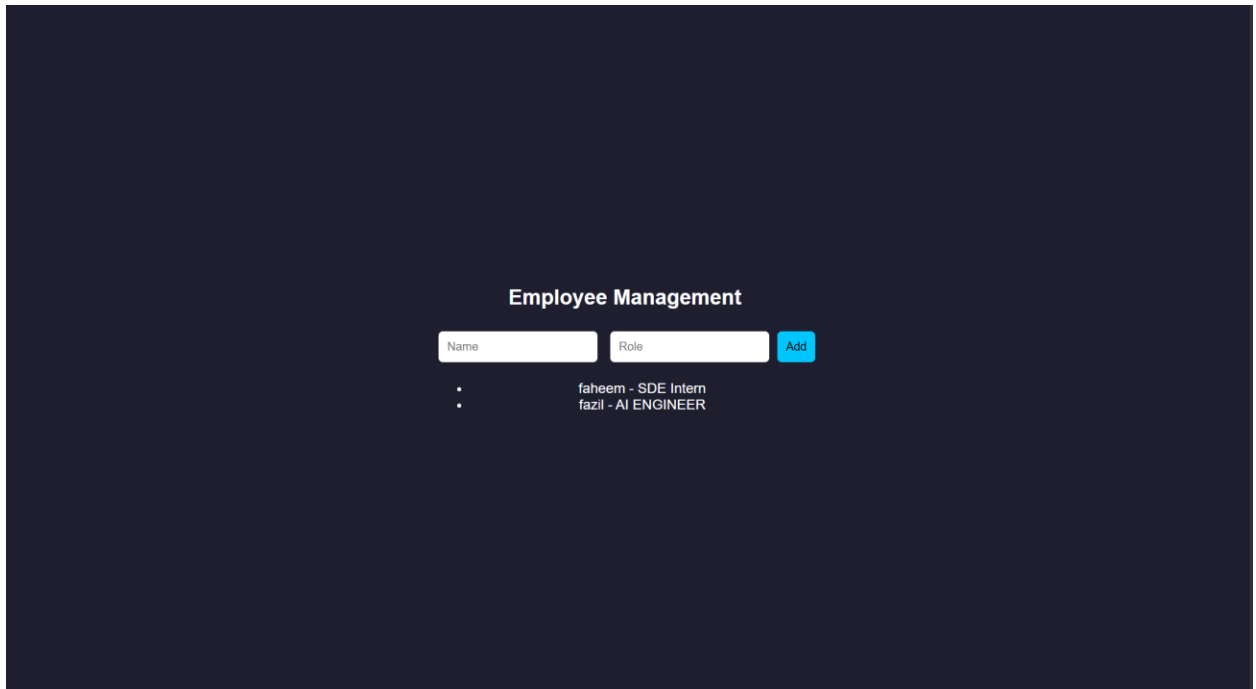
DB.JS:

```

letemployees=[];
functionaddEmployee(emp){
    employees.push(emp);
}
functiongetEmployees(){
    return employees;
}
module.exports={addEmployee,getEmployees};

```

OUTPUT:



The screenshot displays a web application titled "Employee Management" centered on a dark blue background. Below the title, there are two input fields: "Name" and "Role", followed by a blue "Add" button. Below these fields, a list of employees is shown, with each entry consisting of a bullet point, a name, and a role. The first entry is "faheem - SDE Intern" and the second is "fazi - AI ENGINEER".

Name	Role
faheem	SDE Intern
fazi	AI ENGINEER

RESULT:TheEmployeeManagementsystemwassuccessfullyimplementedwithauser-friendly interface and dynamic data handling using Node.js backend.

TASK9:EmployeeMVCAPP

AIM: To develop a basic MVC-based Employee application using HTML, CSS, JavaScript, and Node.js to handle user requests and display employee details.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI form to accept employee details
4. Apply CSS styling for better user interface
5. Capture user input using JavaScript
6. Send request to server using fetch API
7. Create controller logic in server.js
8. Define routes to handle requests (/add, /all)
9. Store employee data in memory using db.js
10. Retrieve employee data from database module
11. Send response back to frontend (view)
12. Display employee details dynamically and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

  <title>EmployeeMVC</title>

  <linkrel="stylesheet"href="style.css">

</head>

<body>
```

```
<divclass="container">

  <h2>EmployeeDetails</h2>

  <inputid="name"placeholder="Name">

  <input id="role"placeholder="Role">

  <buttononclick="addEmployee()">Add</button>

  <ulid="list"></ul>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```
*{

  margin:0;

  padding: 0;

  box-sizing:border-box;

}

body{

  font-family:'SegoeUI',sans-serif;

  background:linear-gradient(135deg,#1f1c2c,#928dab); height:

  100vh;


  display:flex;

  justify-content:center;

  align-items: center;


  color:white;
```

```
}  
  
.container{  
    background:rgba(255,255,255,0.1);  
    padding: 30px;  
    border-radius: 15px;  
    backdrop-filter:blur(10px);  
    width: 320px;  
    text-align:center;  
  
    box-shadow:0 10px 25px rgba(0,0,0,0.4);  
}  
  
h2{  
    margin-bottom:15px;  
}  
  
input{  
    width: 100%;  
    padding:10px;  
    margin:8px 0;  
  
    border: none;  
    border-radius:8px;  
    outline: none;  
  
    transition:0.3s;  
}  
  
input:hover,
```

```
input:focus{  
    transform: scale(1.05);  
    box-shadow: 0 0 8px #00f7ff;  
}
```

```
button{  
    width: 100%;  
    padding: 10px;  
    margin-top: 10px;  
  
    background: #00c6ff;  
    border: none;  
    border-radius: 10px;  
  
    color: white;  
    font-weight: bold;  
    cursor: pointer;  
  
    transition: 0.3s;  
}
```

```
button:hover {  
    background: #0072ff;  
    transform: scale(1.05);  
}
```

```
ul{  
    list-style: none;  
    margin-top: 15px;
```

```

}
li{
  background:rgba(255,255,255,0.15);
  padding: 8px;
  margin: 5px 0;
  border-radius:6px;

  transition:0.3s;
}
li:hover{
  background:rgba(255,255,255,0.3);
}

```

SCRIPT.JS:

```

functionaddEmployee(){
  constname=document.getElementById("name").value;
  const role = document.getElementById("role").value;
  fetch("/add", {
    method:"POST",
    headers:{"Content-Type":"application/json"},
    body: JSON.stringify({ name, role })
  })
  .then(()=>loadEmployees());
}

functionloadEmployees(){
  fetch("/all")
  .then(res=>res.json())

```

```

.then(data=>{
    let list=document.getElementById("list");
    list.innerHTML = "";
    data.forEach(emp=>{
        let li = document.createElement("li");
        li.innerText=emp.name+"-"+emp.role;
        list.appendChild(li);
    });
});
}
loadEmployees();

```

SERVER.JS:

```

const express=require("express");
const cors = require("cors");
const path=require("path");
const db = require("./db");
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(__dirname));
app.post("/add", (req, res) => {
    db.addEmployee(req.body);
    res.json({msg:"EmployeeAdded"});
});
app.get("/all", (req, res) => {
    res.json(db.getEmployees());

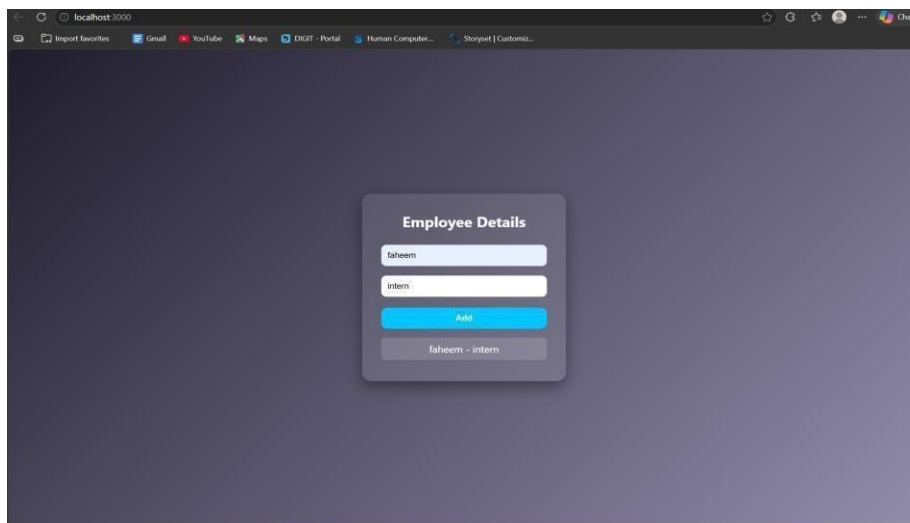
```

```
});  
app.listen(3000, () => {  
  console.log("http://localhost:3000");  
});
```

DB.JS:

```
let employees=[];  
function addEmployee(emp){  
  employees.push(emp);  
}  
function getEmployees(){  
  return employees;  
}  
module.exports={addEmployee,getEmployees};
```

OUTPUT:



RESULT:TheMVC-basedEmployeeapplicationwassuccessfullyimplementedusingNode.js, where user requests were processed and employee details were displayed dynamically.

TASK10: StudentCrudApp

AIM: To develop a Student CRUD Application using HTML, CSS, JavaScript, and Node.js to perform create, read, update, and delete operations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student input (id, name, course)
4. Apply CSS for styling and layout
5. Capture user input using JavaScript
6. Send data to server using fetch API
7. Create backend using Node.js and Express
8. Implement Create operation (POST request)
9. Implement Read operation (GET request)
10. Implement Update operation (PUT request)
11. Implement Delete operation (DELETE request)
12. Display updated student data dynamically and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>StudentCRUD</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h2>StudentCRUD</h2>
<inputid="id"placeholder="ID">
<inputid="name"placeholder="Name">
<inputid="course"placeholder="Course">
<buttononclick="addStudent()">Add</button>
<buttononclick="updateStudent()">Update</button>
<ulid="list"></ul>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family:'SegoeUI';
  background:linear-gradient(135deg,#1f1c2c,#928dab);
  height: 100vh;
  display:flex;
  justify-content:center;
  align-items:center;
  color:white;
}
.container{
  background:rgba(255,255,255,0.1);
  padding:20px;
  border-radius:10px;
  text-align:center;
```

```

}

input{
  display:block;
  margin:8pxauto;
  padding:8px;
}

button {
  margin:5px;
  padding:8px;
  background:#00c6ff;
  border:none;
  color:white;
}

```

SCRIPT.JS:

```

functionaddStudent(){
  let data = getInput();
  fetch("/add", {
    method:"POST",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify(data)
  }).then(loadStudents);
}

functionloadStudents(){
  fetch("/all")
  .then(res=>res.json())
  .then(data=>{

```

```

let list = document.getElementById("list");
list.innerHTML = "";
data.forEach(s => {
    let li = document.createElement("li");
    li.innerHTML = `
        ${s.id} - ${s.name} (${s.course})
        <button onclick="deleteStudent(${s.id})">    </button>
    `;
    list.appendChild(li);
});
});
}

function updateStudent() {
    let data = getInput();
    fetch("/update", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data)
    }).then(loadStudents);
}

function deleteStudent(id) {
    fetch(`/delete/${id}`, { method: "DELETE" })
        .then(loadStudents);
}

function getInput() {
    return {

```

```
    id:document.getElementById("id").value,
    name: document.getElementById("name").value,
    course:document.getElementById("course").value
  };
}
loadStudents();
```

SERVER.JS:

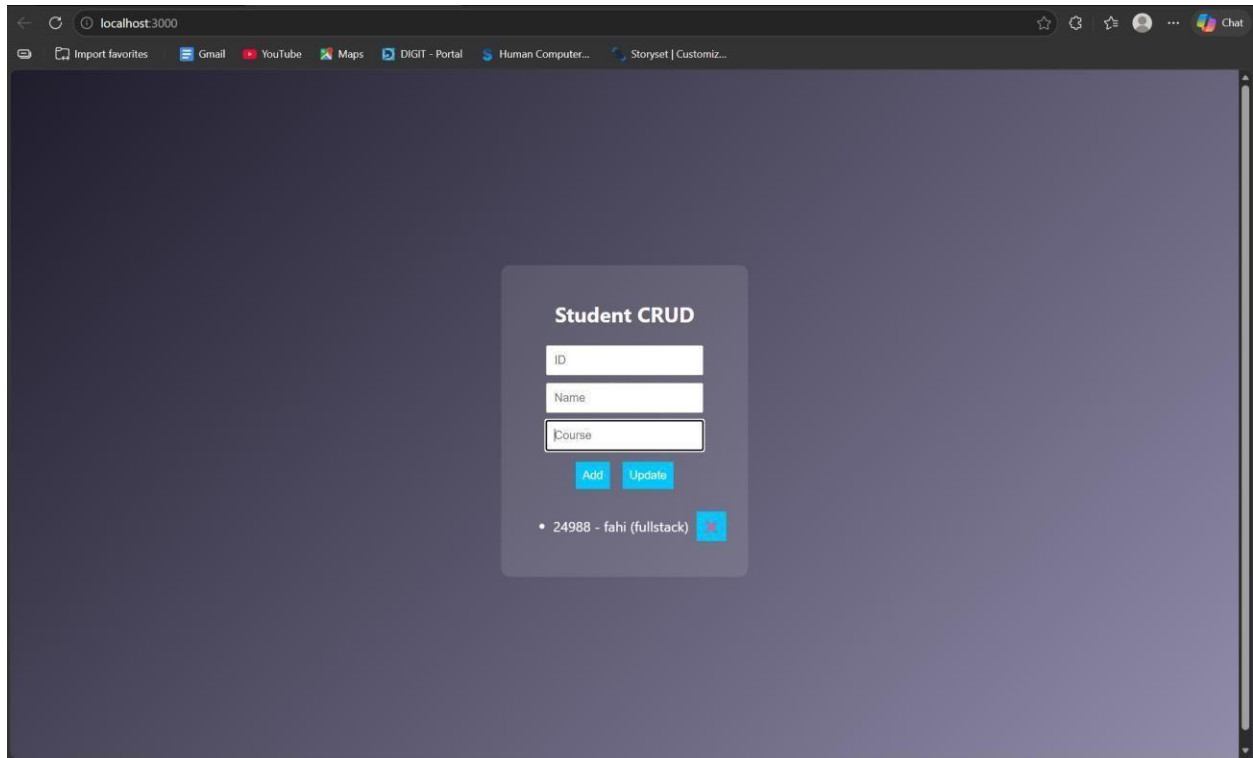
```
const express=require("express");
const cors = require("cors");
const db=require("./db");
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(dirname));
//CREATE
app.post("/add",(req,res)=>{
  db.add(req.body);
  res.json({msg:"Added"});
});
//READ
app.get("/all",(req,res)=>{
  res.json(db.getAll());
});
//UPDATE
app.put("/update",(req,res)=>{
  db.update(req.body);
```

```
    res.json({msg:"Updated"});
  });
//DELETE
app.delete("/delete/:id",(req,res)=> {
    db.remove(req.params.id);
    res.json({msg:"Deleted"});
  });
app.listen(3000,()=>console.log("http://localhost:3000"));
```

DB.JS:

```
letstudents=[];
functionadd(s){
    students.push(s);
}
functiongetAll(){
    return students;
}
function update(updated) {
    students=students.map(s =>
        s.id==updated.id?updated: s
    );
}
functionremove(id){
    students=students.filter(s=>s.id!=id);
}
module.exports={add,getAll,update,remove};
```

OUTPUT:



RESULT:TheStudentCRUDapplicationwassuccessfullyimplementedwithfullcreate,read, update, and delete operations using Node.js and a dynamic web interface.

TASK11:StudentDataLayer

AIM:Toimplementadataaccesslayerformanagingstudentrecordswithfiltering,sorting,and pagination using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Starttheprogram
2. Createprojectfiles(HTML,CSS,JS,server.js,db.js)
3. DesignUIforstudentdetails(id,name,age,department)
4. ApplyCSSstylingforlayoutandUI
5. Capture inputusingJavaScript
6. SenddatatobackendusingfetchAPI
7. CreateNode.jsserverusingExpress
8. Storestudentdatainmemory(db.js)
9. Implementfiltering(bydepartment/age)
10. Implementsorting(byname/age)
11. Implementpagination(limit&pagenumber)
12. Displayfiltered,sorted,paginateddata

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

    <title>StudentDataLayer</title>

    <linkrel="stylesheet"href="style.css">

</head>

<body>
```

```
<divclass="container">

  <h2>StudentData</h2>

  <inputid="name"placeholder="Name">

  <inputid="age" placeholder="Age">

  <inputid="dept"placeholder="Department">

  <buttononclick="addStudent()">Add</button>

  <hr>

  <inputid="filterDept"placeholder="FilterDept">

  <inputid="sortBy"placeholder="Sort(name/age)">

  <inputid="page"placeholder="PageNo">

  <buttononclick="loadStudents()">Search</button>

  <ulid="list"></ul>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```
body{

  font-family: Arial;

  background:#1e1e2f;

  color: white;

  display:flex;

  justify-content:center;

  align-items:center;

  height:100vh;

}
```

```
.container{  
    text-align:center;  
}
```

```
input{  
    display:block;  
    margin:5px;  
    padding:8px;  
}
```

SCRIPT.JS:

```
functionaddStudent(){  
    const data = {  
        name:val("name"),  
        age:val("age"),  
        dept:val("dept")  
    };  
    fetch("/add", {  
        method:"POST",  
        headers:{"Content-Type":"application/json"},  
        body: JSON.stringify(data)  
    }).then(loadStudents);  
}
```

```
functionloadStudents(){  
    constdept=val("filterDept");  
    const sort = val("sortBy");  
    const page = val("page");
```

```

fetch(`/students?dept=${dept}&sort=${sort}&page=${page}`)
  .then(res=>res.json())
  .then(data=>{
    let list=document.getElementById("list"); list.innerHTML
    = "";
    data.forEach(s=>{
      let li = document.createElement("li");
      li.innerText=`${s.name}(${s.age})-${s.dept}`;
      list.appendChild(li);
    });
  });
}

function val(id){
  return document.getElementById(id).value;
}

loadStudents();

```

SERVER.JS:

```

const express=require("express");
const cors = require("cors");
const db=require("./db");
const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(dirname));

// Add student
app.post("/add",(req,res)=>{

```

```
    db.add(req.body);
    res.json({msg:"Added"});
  });
  // Filter + Sort + Pagination
  app.get("/students",(req,res)=>{
    let{dept,sort,page}=req.query;
    let data = db.getAll();
    //FILTER
    if(dept){
      data=data.filter(s=>s.dept===dept);
    }
    //SORT
    if (sort === "name") {
      data.sort((a,b)=>a.name.localeCompare(b.name));
    } else if (sort === "age") {
      data.sort((a,b)=>a.age-b.age);
    }
    //PAGINATION
    letlimit=3;
    page=parseInt(page)||1;
    letstart=(page-1)*limit;
    letpaginated=data.slice(start,start+limit);
    res.json(paginated);
  });
  app.listen(3000,()=>console.log("http://localhost:3000"));
```

DB.JS:

```
let students=[];  
function add(s){  
    students.push(s);  
}  
function getAll(){  
    return students;  
}  
module.exports={add,getAll};
```

OUTPUT:

The screenshot shows a web application titled "Student Data" on a dark background. It features two forms. The first form, at the top, has three input fields with the values "faheem", "21", and "cse", followed by an "Add" button. The second form, below a horizontal separator, has three input fields with the values "cse", "21", and "1", followed by a "Search" button. Below the forms, a list displays the result of the search: "faheem (21) - cse".

RESULT: The data access layer was successfully implemented with filtering, sorting, and pagination using Node.js and dynamic frontend interaction.

TASK12:ProductApi

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

    <title>ProductAPI</title>

</head>

<body>

    <h2>ProductAPITest</h2>

    <input id="id" placeholder="ID">
```

```
<input id="name" placeholder="Name">
<input id="price" placeholder="Price">
<button onclick="add()">Add</button>
<button onclick="get()">GetAll</button>
<ul id="list"></ul>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
/*Reset */
*{
    margin:0;
    padding: 0;
    box-sizing:border-box;
}
/*Background*/
body {
    font-family:'SegoeUI',sans-serif;
    background:linear-gradient(135deg,#141e30,#243b55); height:
    100vh;
    display:flex;
    justify-content:center;
    align-items: center;
    color: white;
}
/*Container*/
```

```
.container{
  background:rgba(255,255,255,0.1);
  padding: 25px;
  border-radius: 15px;
  backdrop-filter:blur(10px);

  width: 350px;
  text-align:center;
  box-shadow:0 10px 25px rgba(0,0,0,0.4);
}

/*Heading*/
h2 {
  margin-bottom:15px;
}

/*Inputs*/
input {
  width: 100%;
  padding:10px;
  margin: 8px 0;
  border: none;
  border-radius:8px;
  outline: none;
  transition: 0.3s;
}

/*Hover+focus effect*/
input:hover,
```

```
input:focus{  
    transform: scale(1.05);  
    box-shadow:008px#00f7ff;  
}
```

```
/*Buttons*/
```

```
button {  
    width: 100%;  
    padding: 10px;  
    margin-top:8px;  
    background:#00c6ff;  
    border: none;  
    border-radius:10px;  
    color: white;  
    font-weight:bold;  
    cursor: pointer;  
    transition: 0.3s;  
}
```

```
/*Buttonhover*/
```

```
button:hover {  
    background:#0072ff;  
    transform:scale(1.05);  
}
```

```
/*ProductList*/
```

```
ul {  
    list-style: none;  
    margin-top:15px;
```

```
}  
  
/*Listitems*/  
li {  
    background:rgba(255,255,255,0.15);  
    padding: 10px;  
    margin: 6px 0;  
    border-radius:8px;  
    display: flex;  
    justify-content:space-between;  
    align-items: center;  
    transition:0.3s;  
}  
  
/*Hovereffect*/  
li:hover {  
    background:rgba(255,255,255,0.3);  
}  
  
/*Optionaldeletebuttoninsidelist*/ li  
button {  
    width: auto;  
    padding: 5px 8px;  
    font-size: 12px;  
    background:#ff4b5c;  
}  
  
li button:hover {  
    background:#ff1e38;  
}
```

SCRIPT.JS:

```
function add() {  
    fetch("/products",{  
        method:"POST",  
        headers:{"Content-Type":"application/json"},  
        body: JSON.stringify(getData())  
    }).then(get);  
}  
  
function get() {  
    fetch("/products")  
    .then(res=>res.json())  
    .then(data=>{  
        letlist=document.getElementById("list");  
        list.innerHTML = "";  
        data.forEach(p=>{  
            let li = document.createElement("li");  
            li.innerText=`${p.id} - ${p.name} - ₹${p.price}`;  
            list.appendChild(li);  
        });  
    });  
}  
  
functiongetData(){  
    return {  
        id:document.getElementById("id").value,  
        name:document.getElementById("name").value,  
        price: document.getElementById("price").value
```

```
};  
}
```

SERVER.JS:

```
const express = require("express");  
const cors = require("cors");  
const db = require("./db");  
const app = express();  
app.use(cors());  
app.use(express.json());  
  
// GET all products  
app.get("/products", (req, res) => {  
  res.json(db.getAll());  
});  
  
// POST add product  
app.post("/products", (req, res) => {  
  db.add(req.body);  
  res.json({ message: "Product added" });  
});  
  
// PUT update product  
app.put("/products/:id", (req, res) => {  
  db.update(req.params.id, req.body);  
  res.json({ message: "Product updated" });  
});  
  
// DELETE product  
app.delete("/products/:id", (req, res) => {
```

```
    db.remove(req.params.id);
    res.json({ message:"Productdeleted"});
  });
// Server start
app.listen(3000,()=>{
  console.log("•   APIrunningathttp://localhost:3000/products");
});
```

DB.JS:

```
let products =[];
functiongetAll(){
  returnproducts;
}
function add(p) {
  products.push(p);
}
function update(id, newData) {
  products=products.map(p=>
    p.id==id?{...p,...newData}:p
  );
}
functionremove(id){
  products=products.filter(p=>p.id!=id);
}
module.exports={getAll,add,update,remove};
```

OUTPUT:

```
Pretty-print ✓  
[  
  {  
    "id": 1,  
    "name": "Phone",  
    "price": 20000  
  },  
  {  
    "id": 2,  
    "name": "Laptop",  
    "price": 50000  
  }  
]
```

RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK13:PRODUCT-API

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

    <title>ProductAPI</title>

</head>

<body>

    <h2>ProductAPITest</h2>

    <input id="id" placeholder="ID">
```

```
<input id="name" placeholder="Name">
<input id="price" placeholder="Price">
<button onclick="add()">Add</button>
<button onclick="get()">GetAll</button>
<ul id="list"></ul>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
/*Reset */
*{
    margin:0;
    padding: 0;
    box-sizing:border-box;
}
/*Background*/
body {
    font-family:'SegoeUI',sans-serif;
    background:linear-gradient(135deg,#141e30,#243b55); height:
    100vh;
    display:flex;
    justify-content:center;
    align-items: center;
    color: white;
}
/*Container*/
```

```
.container{
    background:rgba(255,255,255,0.1);
    padding: 25px;
    border-radius: 15px;
    backdrop-filter:blur(10px);

    width: 350px;
    text-align:center;
    box-shadow:0 10px 25px rgba(0,0,0,0.4);
}

/*Heading*/
h2 {
    margin-bottom:15px;
}

/*Inputs*/
input {
    width: 100%;
    padding:10px;
    margin: 8px 0;
    border: none;
    border-radius:8px;
    outline: none;
    transition: 0.3s;
}

/*Hover+focus effect*/
input:hover,
```

```
input:focus{  
    transform: scale(1.05);  
    box-shadow:008px#00f7ff;  
}
```

```
/*Buttons*/
```

```
button {  
    width: 100%;  
    padding: 10px;  
    margin-top:8px;  
    background:#00c6ff;  
    border: none;  
    border-radius:10px;  
    color: white;  
    font-weight:bold;  
    cursor: pointer;  
    transition: 0.3s;  
}
```

```
/*Buttonhover*/
```

```
button:hover {  
    background:#0072ff;  
    transform:scale(1.05);  
}
```

```
/*ProductList*/
```

```
ul {  
    list-style: none;  
    margin-top:15px;
```

```
}  
  
/*Listitems*/  
li {  
    background:rgba(255,255,255,0.15);  
    padding: 10px;  
    margin: 6px 0;  
    border-radius:8px;  
    display: flex;  
    justify-content:space-between;  
    align-items: center;  
    transition:0.3s;  
}  
  
/*Hovereffect*/  
li:hover {  
    background:rgba(255,255,255,0.3);  
}  
  
/*Optionaldeletebuttoninsidelist*/ li  
button {  
    width: auto;  
    padding: 5px 8px;  
    font-size: 12px;  
    background:#ff4b5c;  
}  
  
li button:hover {  
    background:#ff1e38;  
}
```

SCRIPT.JS:

```
function add() {  
    fetch("/products",{  
        method:"POST",  
        headers:{"Content-Type":"application/json"},  
        body: JSON.stringify(getData())  
    }).then(get);  
}  
  
function get() {  
    fetch("/products")  
    .then(res=>res.json())  
    .then(data=>{  
        letlist=document.getElementById("list");  
        list.innerHTML = "";  
        data.forEach(p=>{  
            let li = document.createElement("li");  
            li.innerText=`${p.id} - ${p.name} - ₹${p.price}`;  
            list.appendChild(li);  
        });  
    });  
}  
  
functiongetData(){  
    return {  
        id:document.getElementById("id").value,  
        name:document.getElementById("name").value,  
        price: document.getElementById("price").value
```

```
};  
}
```

SERVER.JS:

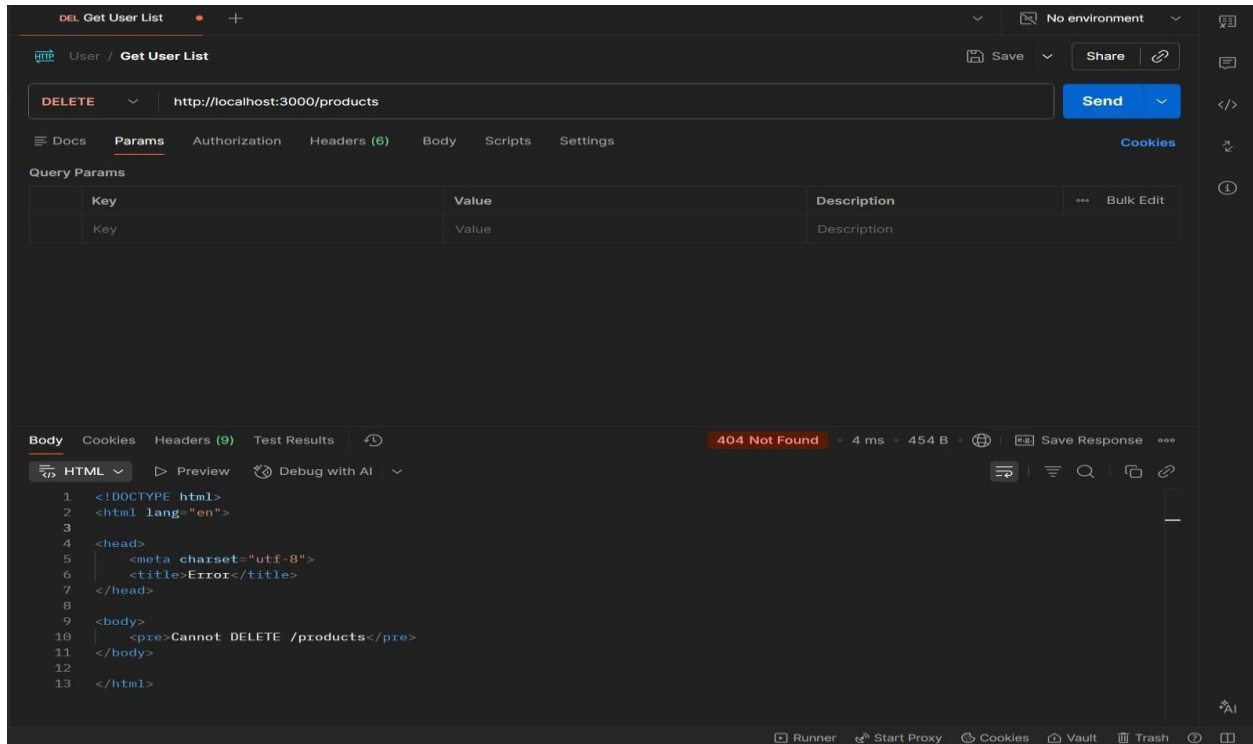
```
const express = require("express");  
const cors = require("cors");  
const db = require("./db");  
const app = express();  
app.use(cors());  
app.use(express.json());  
  
// GET all products  
app.get("/products", (req, res) => {  
    res.json(db.getAll());  
});  
  
// POST add product  
app.post("/products", (req, res) => {  
    db.add(req.body);  
    res.json({ message: "Product added" });  
});  
  
// PUT update product  
app.put("/products/:id", (req, res) => {  
    db.update(req.params.id, req.body);  
    res.json({ message: "Product updated" });  
});  
  
// DELETE product  
app.delete("/products/:id", (req, res) => {
```

```
    db.remove(req.params.id);
    res.json({message:"Productdeleted"});
  });
// Server start
app.listen(3000,()=>{
  console.log("•   APIrunningathttp://localhost:3000/products");
});
```

DB.JS:

```
let products =[];
functiongetAll(){
  returnproducts;
}
function add(p) {
  products.push(p);
}
function update(id, newData) {
  products=products.map(p=>
    p.id==id?{...p,...newData}:p
  );
}
functionremove(id){
  products=products.filter(p=>p.id!=id);
}
module.exports={getAll,add,update,remove};
```

OUTPUT:



RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK14:MICROSERVICESAPP

AIM: To design a simple microservices-based system by splitting a monolithic application into independent services with loose coupling and single responsibility.

ALGORITHM:

1. Start the program
2. Identify modules in monolithic system
3. Split system into independent services
4. Create separate folders for each service
5. Implement Service 1 (Product Service)
6. Implement Service 2 (Order Service)
7. Use separate databases (db.js) for each service
8. Assign different ports for each service
9. Enable communication via REST APIs
10. Keep services loosely coupled
11. Deploy and run services independently
12. Access services and verify functionality

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>MicroservicesApp</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h2>MicroservicesDemo</h2>

<inputid="pname" placeholder="ProductName">

<buttononclick="addProduct()">AddProduct</button>

<inputid="order"placeholder="OrderItem">

<buttononclick="addOrder()">PlaceOrder</button>

</div>

<scriptsrc="script.js"></script>

</body>

</html>
```

STYLE.CSS:

```
body{

    font-family: Arial;

    background:#1e1e2f;

    color: white;

    display:flex;

    justify-content:center;

    align-items:center;

    height:100vh;

}

.container{

    text-align:center;

}

input{

    margin: 5px;

    padding:8px;

}
```

```
button {  
    padding:8px;  
    margin: 5px;  
    background:#00c6ff;  
    border: none;  
}
```

SCRIPT.JS:

```
function addProduct() {  
    fetch("http://localhost:3001/products",{  
        method:"POST",  
        headers:{"Content-Type":"application/json"},  
        body: JSON.stringify({ name: val("pname") })  
    });  
}  
  
function addOrder() {  
    fetch("http://localhost:3002/orders",{  
        method:"POST",  
        headers:{"Content-Type":"application/json"},  
        body: JSON.stringify({ item: val("order") })  
    });  
}  
  
function val(id) {  
    returndocument.getElementById(id).value;  
}
```

SERVER.JS:

```
const express=require("express");
```

```

const cors = require("cors");
const db = require("../db");
const app = express();
app.use(cors());
app.use(express.json());
// GET products
app.get("/products", (req, res) => {
  res.json(db.getAll());
});
// ADD product
app.post("/products", (req, res) => {
  db.add(req.body);
  res.json({ msg: "Product added" });
});
app.listen(3001, () => {
  console.log("Product Service running at http://localhost:3001");
});

```

DB.JS:

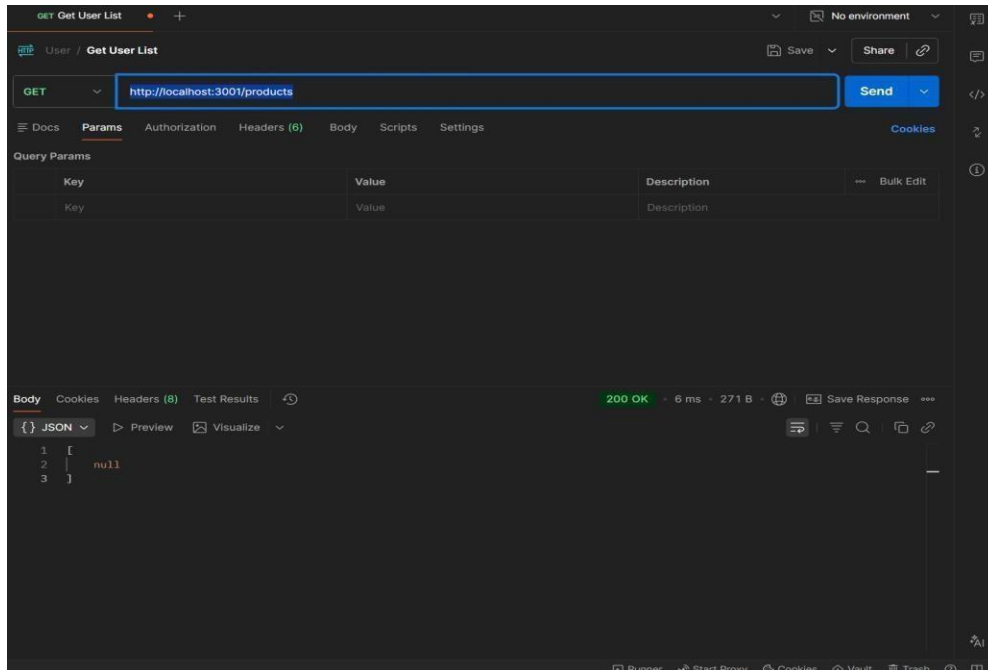
```

let products = [];
function add(p) {
  products.push(p);
}
function getAll() {
  return products;
}
module.exports = { add, getAll };

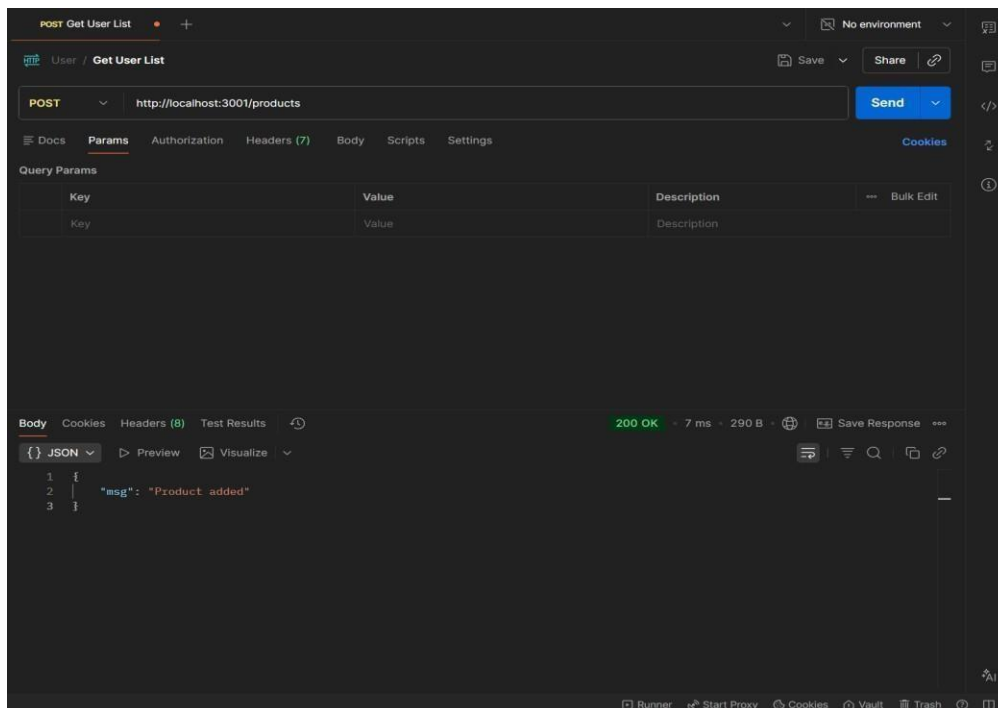
```

OUTPUT:

1. GET:<http://localhost:3001/products>



2. POST:<http://localhost:3001/products>



RESULTS:The application was successfully divided into independent microservices with separate responsibilities and communication via REST APIs

TASK15:ServiceRegistryApp

Aim:To implementservice registry and discovery using Node.js by dynamically registering and discovering microservices without hard-coded URLs.

ALGORITHM:

1. Start the program
2. Create a registry server
3. Store service name and URL in memory
4. Start Product Service
5. Register Product Service to registry
6. Start Order Service
7. Register Order Service to registry
8. Maintain service list in registry
9. Client requests service from registry
10. Fetch service URL dynamically
11. Call the discovered service API
12. Stop the program

PROGRAM:

SERVER.JS:

```
const express = require("express");  
  
const app = express();  
  
app.use(express.json());  
  
let services = {};  
  
app.post("/register", (req, res) => {  
    const { name, url } = req.body; services[name]  
    = url;
```

```

    console.log(`${name}registeredat${url}`);

    res.json({ msg: "Registered" });
  });

app.get("/service/:name", (req, res) => {

  constservice=services[req.params.name]; if

  (!service) {

    returnres.status(404).json({error:"Service not found"})

    res.json({ url: service });

  });

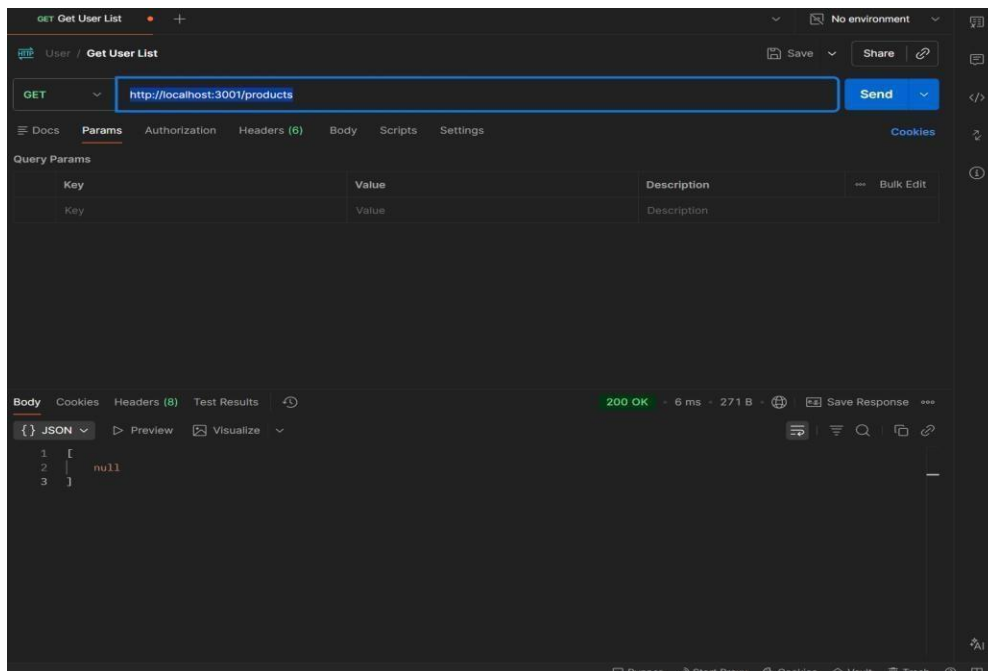
app.listen(4000,()=>{

  console.log("Registryrunningathttp://localhost:4000");

});

```

OUTPUT



RESULT: The microservices-based system was successfully implemented with service registry and discovery, enabling dynamic communication between independent services without hard-coded URLs.

TASK16:APIGATEWAYAPI

AIM:ToconfigureanAPIGatewaythatroutesclientrequeststomicroservicesandperforms basic load balancing across multiple service instances.

ALGORITHM:

1. Starttheprogram
2. CreateAPIGatewayserver
3. Defineroutesformicroservices
4. Createmultipleinstancesofaservice
5. StoreserviceURLsinanarray
6. Implementround-robinloadbalancing
7. Receiveclientrequestatgateway
8. Selectserviceinstancedynamically
9. Forwardrequesttoselectedinstance
10. Getresponsefromservice
11. Sendresponsebacktoclient
12. Stoptheprogram

PROGRAM:

SERVER.JS:

```
const express=require("express");  
const axios = require("axios");  
const app = express();  
const services = [  
  "http://localhost:3001",  
  "http://localhost:3002"  
];
```

```

let index=0;
app.get("/products",async(req,res)=>{
  try {
    const service = services[index];
    index=(index+1)%services.length;
    const response= await axios.get(service + "/products");
    res.json(response.data);

  }catch(err){
    res.status(500).json({error:"Service unavailable"});
  }
});
app.listen(5000,()=>{
  console.log("APIGateway→http://localhost:5000/products");
});

```

OUTPUT:

```

{
  "instance": "Service 1",
  "data": ["Laptop", "Phone"]
}

```

```

{
  "instance": "Service 2",
  "data": ["Tablet", "Watch"]
}

```

RESULT: The API Gateway was successfully implemented to route requests and distribute load across multiple service instances using round-robin load balancing.

TASK17:InterServiceApp

AIM:To enable inter-service communication where one microservice consumes another service's REST API and handles responses and failures gracefully.

ALGORITHM:

1. Start the program
2. Create ProductService(provider)
3. Create OrderService(consumer)
4. Implement REST API in ProductService
5. Store product data in memory
6. Implement OrderService API
7. Use axios/fetch to call ProductService
8. Receive response from ProductService
9. Process and store order data
10. Handle errors using try-catch
11. Send proper success/error response
12. Stop the program

FOLDER STRUCTURE:

inter-service-app/

|

├── product-service/

| └─ server.js

|

├── order-service/

| └─ server.js

PROGRAM:

PRODUCT-SERVICE:SERVER.JS:

```
const express = require("express");

const app = express();

let products = [
  {id:1,name:"Laptop"},
  {id:2,name:"Phone"}
];

app.get("/products",(req,res)=>{
  res.json(products);
});

app.listen(3001,()=>{
  console.log("ProductService→http://localhost:3001/products");
});
```

ORDER-SERVICE:SERVER.JS:

```
const express = require("express");

const axios = require("axios");

const app = express();

app.use(express.json());

let orders = [];

app.post("/orders",async(req,res)=>{
  try {
    const response = await axios.get("http://localhost:3001/products");const
    products = response.data;

    const order = {
      id:Date.now(),
```

```

        products:products
    };
    orders.push(order);
    res.json({
        message:"Ordercreatedsuccessfully",
        order: order
    });
} catch (error) {
    res.status(500).json({
        error:"FailedtofetchproductsfromProductService"
    });
}
});
app.get("/orders",(req,res)=>{
    res.json(orders);
});
app.listen(3002,()=>{
    console.log("OrderService→http://localhost:3002/orders");
});

```

OUTPUT:

```

{
  "message": "Order created successfully",
  "order": {
    "id": 123456,
    "products": [
      { "id": 1, "name": "Laptop" },
      { "id": 2, "name": "Phone" }
    ]
  }
}

```

```
1  [  
2    {  
3      "id": 1,  
4      "name": "Laptop"  
5    },  
6    {  
7      "id": 2,  
8      "name": "Phone"  
9    }  
10 ]
```

RESULT: Inter-service communication was successfully implemented using REST APIs, with proper handling of responses and failures between microservices.

TASK18:MicroserviceTest

AIM:Towriteunittestcasesformicroservicestovalidateservicelogic,RESTAPIs,anddata handling using a testing framework.

ALGORITHM:

1. Starttheprogram
2. Installtestingframework(Jest, Supertest)
3. Configuretestscriptinpackage.json
4. Createtestfolder/files
5. Importservicemodules
6. Writetestcasesforservicelogic
7. Writetestcases forRESTAPIs
8. Mockexternaldependenciesifneeded
9. ExecutetestcasesusingJest
10. Verifyexpectedoutputs
11. Handleedgecasesanderrors
12. Stoptheprogram

PROGRAM:

app.js:

```
const express = require("express");  
const app = express();  
app.use(express.json());  
let products = [];  
  
// API  
app.get("/products", (req, res) => {  
    res.json(products);  
});
```

```
app.post("/products", (req,res)=> {  
  products.push(req.body);  
  res.json({msg:"Productadded"});  
});  
module.exports=app;
```

SERVER.JS:

```
const app=require("./app");  
app.listen(3001, () => {  
  console.log("ProductService running");  
});
```

Product.Test.js:

```
const request=require("supertest");  
const app=require("../product-service/app");  
describe("Product API Tests", () => {  
  test("GET/products should return empty array", async()=>{  
    const res = await request(app).get("/products");  
    expect(res.statusCode).toBe(200);  
    expect(res.body).toEqual([]);  
  });  
  test("POST/products should add product", async()=>{ const  
    res = await request(app)  
      .post("/products")  
      .send({ name: "Laptop" });  
    expect(res.statusCode).toBe(200);  
    expect(res.body.msg).toBe("Productadded");  
  });  
  test("GET/products should return data", async()=>{
```

```
const res = await request(app).get("/products");  
expect(res.body.length).toBeGreaterThan(0);  
});  
});
```

OUTPUT:

```
PASS tests/product.test.js  
✓ GET /products should return empty array  
✓ POST /products should add product  
✓ GET /products should return data
```

RESULT: Unit test cases were successfully implemented to validate microservice logic, REST APIs, and data handling, ensuring application reliability.

TASK19:MiniGitSystem

AIM:TodesignasimpleGit-likewebapplicationthattracksfilechanges,storesversions,and displays commit history using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Starttheprogram
2. Createprojectfiles(HTML,CSS,JS,server.js,db.js)
3. DesignUIforenteringfilecontent
4. CaptureuserinputusingJavaScript
5. SenddatatoserverusingfetchAPI
6. Storeversionsindatabase(array)
7. Assignversionnumbertoeachcommit
8. Savecommitmessageandtimestamp
9. CreateAPItofetchcommithistory
10. DisplaycommithistoryinUI
11. Allowviewingpreviousversions
12. Stoptheprogram

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

  <title>MiniGit System</title>

  <linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="container">
```

```
<h2>MiniGitSystem</h2>
<textareaid="content"placeholder="Writeyourcode..."></textarea>
<inputid="message"placeholder="Commitmessage">
<buttononclick="commit()">Commit</button>
<h3>CommitHistory</h3>
<ulid="history"></ul>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background:#1e1e2f;
  color: white;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}
.container {
  width:400px;
  text-align:center;
}
textarea {
  width:100%;
```

```

height: 80px;
    margin: 5px;
}
input, button {
    margin: 5px;
    padding: 8px;
}
li{
    background: #333;
    margin: 5px;
    padding: 5px;
}

```

SCRIPT.JS:

```

function commit(){
    const content = document.getElementById("content").value;
    const message = document.getElementById("message").value;
    fetch("/commit", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({ content, message })
    }).then(loadHistory);
}

function loadHistory(){
    fetch("/history")
        .then(res => res.json())
        .then(data => {

```

```

    let list = document.getElementById("history");
    list.innerHTML = "";
    data.forEach(c => {
        let li = document.createElement("li");
        li.innerText = `v${c.version}: ${c.message}`;
        list.appendChild(li);
    });
});
}
loadHistory();

```

SERVER.JS:

```

const express = require("express");
const db = require("./db");
const app = express();
app.use(express.json());
app.use(express.static(_dirname));
app.post("/commit", (req, res) => {
    db.save(req.body);
    res.json({ msg: "Committed" });
});
app.get("/history", (req, res) => {
    res.json(db.getAll());
});
app.listen(3000, () => {
    console.log("http://localhost:3000");
});

```

DB.JS:

```
let commits=[];

let version = 1;

function save(data){

    commits.push({

        version:version++,

        message:data.message,

        content: data.content,

        time: new Date()

    });

}

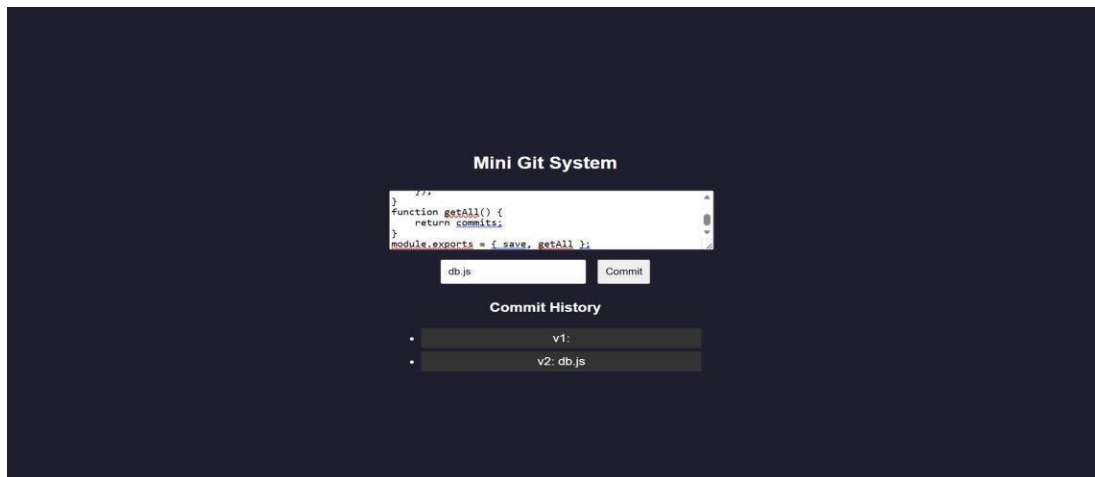
function getAll() {

    return commits;

}

module.exports={save,getAll};
```

OUTPUT:



RESULT: A mini Git-like system was successfully developed to track versions, store commits, and display history through a web interface.

TASK20:MiniGitBranchApp

Aim: To develop a Git-like application that supports branching, merging, rebasing, and conflict resolution using a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Store branches and commits in memory
4. Create default branch (main)
5. Add feature to create new branch
6. Commit changes to selected branch
7. Implement merge logic
8. Detect conflicts (same file modified)
9. Show conflict message
10. Resolve conflicts manually
11. Implement rebase logic
12. Display branch history and stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

  <title>MiniGit Branching</title>

  <linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="container">
```

```
<h2>MiniGitBranching</h2>
<inputid="branchName"placeholder="NewBranchName">
<buttononclick="createBranch()">CreateBranch</button>
<inputid="message"placeholder="CommitMessage">
<buttononclick="commit()">Commit</button>
<buttononclick="merge()">Merge</button>
<buttononclick="rebase()">Rebase</button>
<h3>Branches</h3>
<ulid="branches"></ul>
<h3>History</h3>
<ulid="history"></ul>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background:#1e1e2f;
  color: white;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}
.container{
```

```
    text-align:center;
}
input, button {
    margin: 5px;
    padding:8px;
}
li{
    background:#333;
    margin: 5px;
    padding: 5px;
}
```

SCRIPT.JS:

```
functioncreateBranch(){
    constname=document.getElementById("branchName").value;
    fetch("/branch", {
        method:"POST",
        headers:{"Content-Type":"application/json"},
        body: JSON.stringify({ name })
    }).then(load);
}
functioncommit(){
    constmessage=document.getElementById("message").value;
    fetch("/commit", {
        method:"POST",
        headers:{"Content-Type":"application/json"},
        body: JSON.stringify({ message })
    })
}
```

```

    }).then(load);
}
function merge(){
    fetch("/merge",{method: "POST"})
        .then(res=>res.json())
        .then(alert)
        .then(load);
}
function rebase() {
    fetch("/rebase",{method:"POST"})
        .then(load);
}
function load(){
    fetch("/data")
        .then(res=>res.json())
        .then(data=>{
            let b=document.getElementById("branches");
            let h = document.getElementById("history");
            b.innerHTML = "";
            h.innerHTML = "";
            data.branches.forEach(br=>{
                let li=document.createElement("li");
                li.innerText = br;
                b.appendChild(li);
            });
            data.history.forEach(c=>{

```

```

        let li=document.createElement("li");
        li.innerText = c;
        h.appendChild(li);
    });
});
}
load();

```

SERVER.JS:

```

const express=require("express");
const db = require("./db");
const app = express();
app.use(express.json());
app.use(express.static(_dirname));
app.post("/branch", (req, res) => {
    db.createBranch(req.body.name);
    res.json({msg:"Branchcreated"});
});
app.post("/commit",(req,res)=>{
    db.commit(req.body.message);
    res.json({ msg: "Committed" });
});
app.post("/merge",(req,res)=>{
    const result = db.merge();
    res.json({ msg: result });
});
app.post("/rebase",(req,res) =>{

```

```

    db.rebase();
    res.json({msg:"Rebased"});
  });
  app.get("/data",(req,res)=> {
    res.json(db.getData());
  });
  app.listen(3000,()=>console.log("http://localhost:3000"));

```

DB.JS:

```

let branches=["main"];
let current = "main";
let commits = {
  main:[]
};
function createBranch(name){
  branches.push(name);
  commits[name]=[...commits[current]];
}
function commit(msg) {
  commits[current].push(msg);
}
function merge(){
  let mainCommits=commits["main"];
  let featureCommits=commits[current];
  if(mainCommits.length&&featureCommits.length){ return
    "Merge Conflict Detected!";
  }
}

```

```

commits["main"]=[...mainCommits,...featureCommits];

    return "Merged Successfully";
}

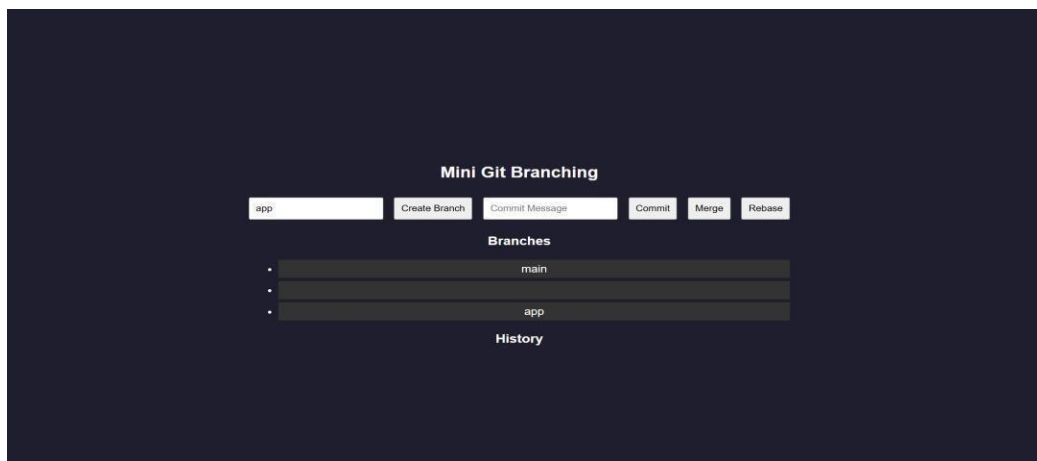
function rebase() {
    commits[current]= [...commits["main"],...commits[current]];
}

function getData(){
    return {
        branches,
        history:commits[current]
    };
}

module.exports={createBranch,commit,merge, rebase,getData};

```

OUTPUT:



RESULT: A Git-like branching application was successfully developed to simulate branching, merging, rebasing, and conflict resolution through a web interface.

TASK21:MiniCicdApp

AIM:TodevelopaCI/CDpipelinesimulationapplicationthatautomatesbuild,test,and deployment steps through a web interface.

ALGORITHM:

1. Starttheprogram
2. Createprojectfiles(HTML,CSS,JS,server.js,db.js)
3. DesignUIforpipelinetrigger
4. Captureuseraction(RunPipeline)
5. Sendrequesttobackend
6. Simulate codecheckoutstep
7. Simulate buildprocess
8. Simulatetestexecution
9. Simulatedeployment step
10. Storepipelinelogs
11. Displaypipeline statusinUI
12. Stoptheprogram

PROGRAM:

INDEX.HTML:

```
<!DOCTYPEhtml>

<html>

<head>

  <title>Mini CI/CDPipeline</title>

  <linkrel="stylesheet"href="style.css">

</head>

<body>

<divclass="container">
```

```
<h2>CI/CDPipelineSimulator</h2>
<button onclick="runPipeline()">RunPipeline</button>
<h3>PipelineLogs</h3>
<ulid="logs"></ul>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
    font-family: Arial;
    background:#1e1e2f;
    color: white;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}
.container{
    text-align:center;
    width: 400px;
}
button {
    padding:10px;
    margin: 10px;
    background:#00c6ff;
```

```
border: none;
color: white;
cursor:pointer;
}
li{
background:#333;
margin: 5px;
padding: 5px;
}
```

SCRIPT.JS:

```
function runPipeline() {
  fetch("/run",{method:"POST"})
    .then(()=>loadLogs());
}

function loadLogs(){
  fetch("/logs")
    .then(res=>res.json())
    .then(data=>{
      let list=document.getElementById("logs");
      list.innerHTML = "";
      data.forEach(log=>{
        let li=document.createElement("li");
        li.innerText = log;list.appendChild(li);
      });
    });
}
```

```
}
```

```
loadLogs();
```

SERVER.JS:

```
const express = require("express");
```

```
const db = require("./db");
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use(express.static(_dirname));
```

```
app.post("/run", async (req, res) => {
```

```
    await db.runPipeline();
```

```
    res.json({ msg: "Pipeline executed" });
```

```
});
```

```
app.get("/logs", (req, res) => {
```

```
    res.json(db.getLogs());
```

```
});
```

```
app.listen(3000, () => {
```

```
    console.log("http://localhost:3000");
```

```
});
```

DB.JS:

```
let logs = [];
```

```
async function runPipeline() { logs
```

```
    = [];
```

```
    logs.push("    CodeCheckout...");
```

```
    await delay();
```

```
    logs.push("    BuildStarted...");
```

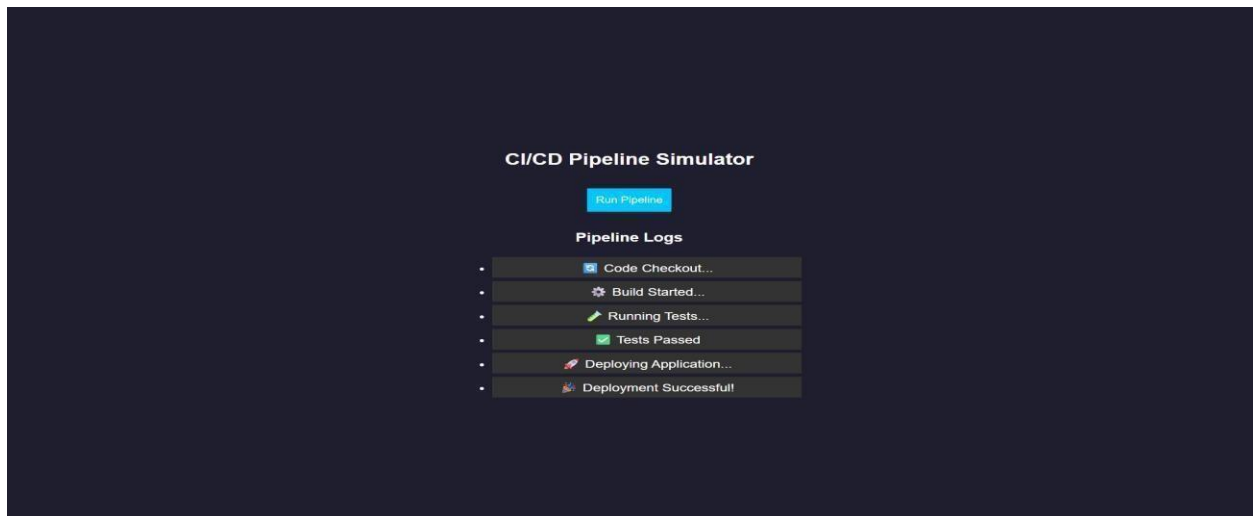
```
    await delay();
```

```

logs.push("    RunningTests...");
await delay();
logs.push("    TestsPassed");
await delay();
logs.push("•    DeployingApplication...");
await delay();
logs.push("    DeploymentSuccessful!");
}
functiongetLogs(){
    return logs;
}
functiondelay(){
    returnnew Promise(resolve=> setTimeout(resolve,1000));
}
module.exports= {runPipeline,getLogs};

```

OUTPUT:



RESULT:ACI/CDpipelinesimulationapplicationwassuccessfullydevelopedtoautomateand visualize build, test, and deployment processes.

TASK22:CloudApp

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation
10. Calculate cost based on pay-as-you-use pricing model
11. Display total cost dynamically on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>CloudComparison</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h2>CloudServiceComparison</h2>
<selectid="provider"onchange="loadData()">
  <optionvalue="aws">AWS</option>
  <optionvalue="gcp">GCP</option>
  <optionvalue="azure">Azure</option>
</select>
<h3>Services</h3>
<ulid="services"></ul>
<h3>Pricing Calculator</h3>
<inputid="hours"placeholder="ComputeHours">
<inputid="storage"placeholder="Storage(GB)">
<buttononclick="calculate()">CalculateCost</button>
<h3id="result"></h3>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background:#1e1e2f;
  color: white;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
```

```

}

.container{
    text-align:center;
    width: 350px;
}

input,select,button{
    margin: 5px;
    padding: 8px;
}

li{
    background:#333;
    margin: 5px;
    padding: 5px;
}

```

SCRIPT.JS:

```

functionloadData() {
    constprovider=document.getElementById("provider").value;
    fetch(`/services/${provider}`)
    .then(res=>res.json())
    .then(data=>{
        letlist=document.getElementById("services");
        list.innerHTML = "";
        Object.entries(data).forEach(([key,value])=>{
            letli= document.createElement("li");
            li.innerText = `${key}: ${value}`;
            list.appendChild(li);

```

```

    });
  });
}
function calculate(){
  const provider = document.getElementById("provider").value;
  const hours = document.getElementById("hours").value;
  const storage = document.getElementById("storage").value;
  fetch("/cost", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({provider, hours, storage})
  })
  .then(res => res.json())
  .then(data => {
    document.getElementById("result").innerText = "Cost: $" + data.cost;
  });
}
loadData();

```

SERVER.JS:

```

const express = require("express");
const db = require("../db");
const app = express();
app.use(express.json());
app.use(express.static(_dirname));
app.get("/services/:provider", (req, res) => {
  res.json(db.getServices(req.params.provider));
});

```

```

});
app.post("/cost", (req,res)=>{
    const{provider,hours,storage}=req.body;
    constcost=db.calculate(provider,hours,storage); res.json({
    cost });
});
app.listen(3000, () => {
    console.log("http://localhost:3000");
});

```

DB.JS:

```

constservices={
    aws: {
        Compute:"EC2",
        Storage: "S3",
        Database:"RDS",
        Network: "VPC"
    },
    gcp:{
        Compute:"ComputeEngine",
        Storage: "Cloud Storage",
        Database: "Cloud SQL",
        Network: "VPC"
    },
    azure:{
        Compute:"VirtualMachines",
        Storage: "Blob Storage",

```

```

    Database:"SQLDatabase",
    Network: "VNet"
  }
};

const pricing={
  aws: { compute: 0.1, storage: 0.02 },
  gcp: { compute: 0.09, storage: 0.025 },
  azure:{compute:0.11,storage:0.018}
};

function getServices(provider){
  return services[provider];
}

function calculate(provider, hours, storage){
  const p = pricing[provider];
  return (hours*p.compute+storage*p.storage).toFixed(2);
}

module.exports={getServices,calculate};

```

OUTPUT:

The screenshot shows a web application titled "Cloud Service Comparison" on a dark background. At the top, there is a dropdown menu set to "AWS". Below this, a section titled "Services" lists four items: "Compute: EC2", "Storage: S3", "Database: RDS", and "Network: VPC", each preceded by a small blue square icon. Underneath the services list is a "Pricing Calculator" section. It features a text input field containing the number "9", a smaller text input field containing the number "8", and a button labeled "Calculate Cost". At the bottom of the calculator section, the result is displayed as "Cost: \$1.06".

Cloud Service Comparison

GCP

Services

- Compute: Compute Engine
- Storage: Cloud Storage
- Database: Cloud SQL
- Network: VPC

Pricing Calculator

9

8

Calculate Cost

Cost: \$1.01

Cloud Service Comparison

Azure

Services

- Compute: Virtual Machines
- Storage: Blob Storage
- Database: SQL Database
- Network: VNet

Pricing Calculator

9

8

Calculate Cost

Cost: \$1.13

RESULTS:Acloudcomparisonapplicationwassuccessfullydevelopedtodisplayservicesand simulate pay-as-you-use pricing across AWS, GCP, and Azure.

TASK23: AIPROMPTAPP

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API
6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code/cloud config/response)
9. Evaluate output quality and assign score
10. Send generated output and score to frontend
11. Display output and score on UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>AIPromptApp</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>
```

```
<divclass="container">
  <h2>PromptEngineeringTool</h2>
  <textareaid="prompt"placeholder="Enteryourprompt..."></textarea>
  <buttononclick="generate()">GenerateOutput</button>
  <h3>GeneratedOutput</h3>
  <preid="output"></pre>
  <h3>QualityScore</h3>
  <pid="score"></p>
</div>
<scriptsrc="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background:#202038;
  color: white;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}
.container {
  width:400px;
  text-align:center;
}
```

```
textarea {  
  width: 100%;  
  height:100px;  
  margin: 5px;  
}  
  
button {  
  padding:10px;  
  margin: 10px;  
  background:#00c6ff;  
  border: none;  
  color:white;  
}  
  
pre{  
  background:#333;  
  padding: 10px;  
}
```

SCRIPT.JS:

```
functiongenerate(){  
  constprompt=document.getElementById("prompt").value;  
  fetch("/generate", {  
    method:"POST",  
    headers:{"Content-Type":"application/json"},  
    body: JSON.stringify({ prompt })  
  })  
  .then(res=>res.json())  
  .then(data=>{
```

```
    document.getElementById("output").innerText = data.output;
    document.getElementById("score").innerText=data.score+"/10";
  });
}
```

SERVER.JS:

```
const express = require("express");
const db = require("../db");

const app = express();

// Middleware
app.use(express.json());
app.use(express.static(dirname)); // serve HTML

// Home route (optional but useful)
app.get("/", (req, res) => {
  res.sendFile(dirname + "/index.html");
});

// Generate output
app.post("/generate", (req, res) => {
  const result = db.processPrompt(req.body.prompt);
  res.json(result);
});

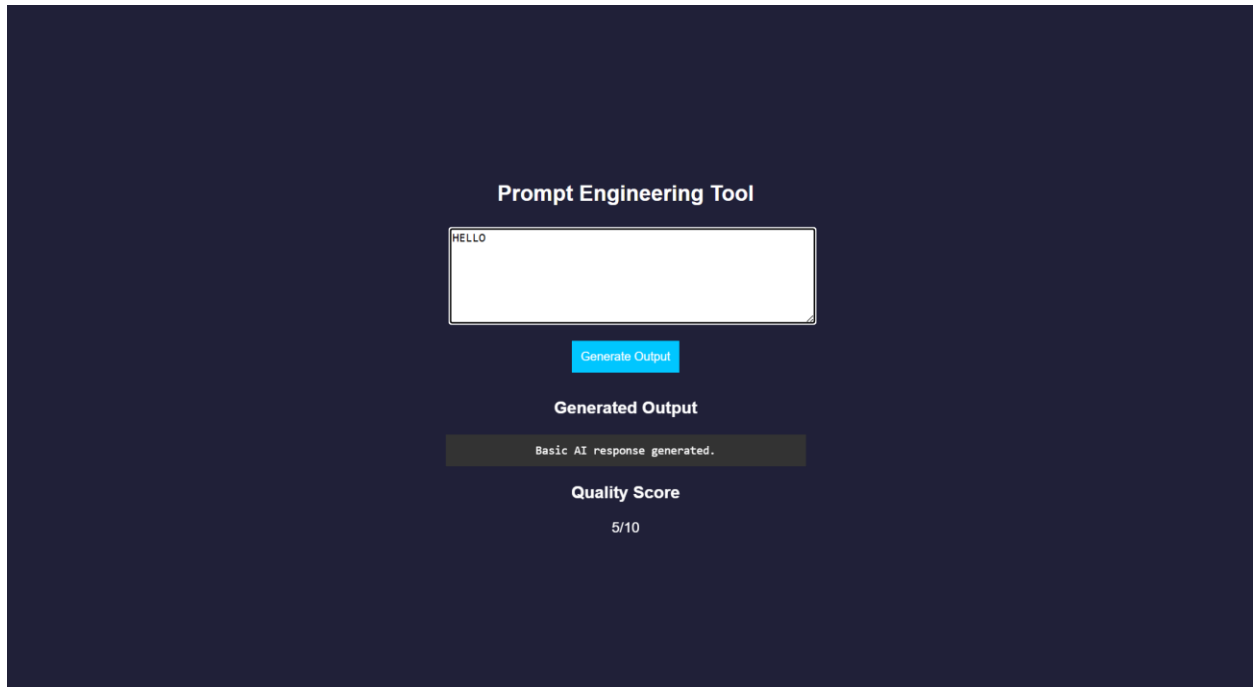
// START SERVER WITH LINK
app.listen(3000, () => {
  console.log("• Server running at:");
```

```
    console.log("http://localhost:3000");  
  });
```

DB.JS:

```
function processPrompt(prompt){  
  let output = "";  
  let score=5;  
  if(prompt.toLowerCase().includes("code")){  
    output = `function greet() {  
      console.log("Hello from AI!");  
    }`;  
    score=9;  
  }  
  elseif(prompt.toLowerCase().includes("cloud")){  
    output = `AWS EC2 Config:  
- Instance: t2.micro  
- Region:us-east-1`;  
    score = 8;  
  }  
  else{  
    output="Basic AI response generated."  
    score = 5;  
  }  
  
  return{output,score};  
}  
  
module.exports={processPrompt};
```

OUTPUT:



The screenshot displays a web application titled "Prompt Engineering Tool" on a dark blue background. It features a central white text input field containing the word "HELLO". Below the input field is a blue button labeled "Generate Output". Underneath the button, the text "Generated Output" is displayed. A dark grey box contains the message "Basic AI response generated.". At the bottom, the text "Quality Score" is shown above a score of "5/10".

RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

TASK24:VibeCloudApp

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API/Cloud/Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Vibe CodingApp</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Vibe Coding Cloud Generator</h2>
```

```
<textarea id="prompt" placeholder="Enter prompt..."></textarea>
<button onclick="generate()">Generate</button>
<h3>Output</h3>
<pre id="output"></pre>
<h3>Score</h3>
<div id="score"></div>
<h3>Prompt Versions</h3>
<ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body{
  font-family: Arial;
  background: #b98bdb;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.container {
  width: 400px;
  text-align: center;
}
```

```
textarea {  
    width: 100%;  
    height:100px;  
}  
button {  
    padding:10px;  
    margin: 10px;  
    background:#00c6ff;  
    border: none;  
}  
pre{  
    background:#333;  
    padding: 10px;  
}  
li{  
    background:#444;  
    margin: 5px;  
    padding: 5px;  
}
```

SCRIPT.JS:

```
functiongenerate(){  
    constprompt=document.getElementById("prompt").value;  
    fetch("/generate", {  
        method:"POST",  
        headers: {"Content-Type":"application/json"},
```

```

        body:JSON.stringify({ prompt})
    })
    .then(res=>res.json())
    .then(data=>{
        document.getElementById("output").innerText = data.output;
        document.getElementById("score").innerText=data.score+"/10"; loadHistory();
    });
}

functionloadHistory(){
    fetch("/history")
    .then(res=>res.json())
    .then(data=>{
        letlist=document.getElementById("history");
        list.innerHTML = "";
        data.forEach((p,i)=>{
            letli=document.createElement("li");
            li.innerText = `v${i+1}: ${p.prompt}`;
            list.appendChild(li);
        });
    });
}

```

loadHistory();

SERVER.JS:

```
constexpress=require("express");
```

```
const db = require("./db");
```

```

const app = express();
app.use(express.json());
app.use(express.static(_dirname));
app.post("/generate",(req,res)=>{
    const result=db.process(req.body.prompt);
    res.json(result);
});
app.get("/history",(req,res)=>{
    res.json(db.getHistory());
});
app.listen(3000,()=>{
    console.log("• http://localhost:3000");
});

```

DB.JS:

```

let history=[];
function process(prompt){
    let output = "";
    let score = 5;
    history.push({prompt});
    if(prompt.includes("REST")){
        output=`app.get('/api',(req,res)=>res.send('API running'))`;
        score = 7;
    }
    elseif(prompt.includes("cloud")){
        output=`Deploy using AWS EC2:

```

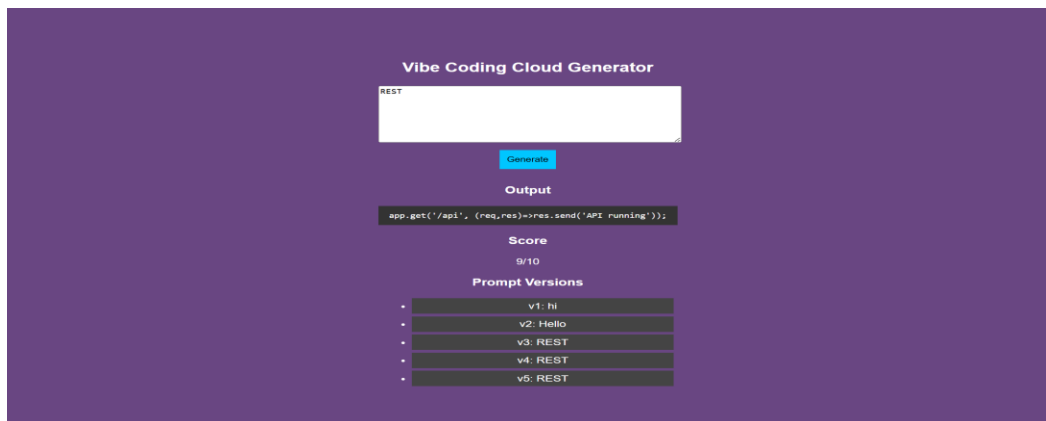
1. Launch instance

2. Install Node.js

3. Runapp`;

```
    score=8;
  }
  elseif(prompt.includes("security")){
    output=`UseHTTPS,JWTAuthentication,andfirewallrules`; score
    = 9;
  }
  score+=Math.min(history.length,2);
  return { output, score };
}
functiongetHistory(){
  return history;
}
module.exports= {process,getHistory};
```

OUTPUT:



RESULT:AVibeCodingapplicationwassuccessfullydevelopedtogenerateandrefinecloud-based features using iterative prompt engineering.